

an opinionated way to start programming competitively

ทศพร แสงจำ

แนวทางเริ่มเขียนโปรแกรมแข่งขัน

Contents

0.1. หลักการ	8
0.1.1. การออกแบบวิธีการคิด	8
0.1.2. การเขียนโปรแกรมตามวิธี	8
0.2. การบริหารเวลา	8
0.3. Time Complexity	8
1. Common STL library	10
1.1. Max/Min	10
1.2. Sort	10
1.3. Vector	10
1.3.1. Access	10
1.3.2. Functions	10
1.3.3. Loop	10
1.3.4. Example	10
1.4. Pair	11
1.4.1. Access	11
1.4.2. Example	11
2. CPP Syntax for Competitive Programming	12
2.1. Bare Minimum	12
2.1.1. Code Template	12
2.1.2. Comments	12
2.1.3. Variables Data Types	12
2.1.4. Input/Output	12
2.1.5. Control Structures	14
2.1.5.1. condition	14
2.1.5.2. loop	14
2.1.6. Arrays	14
2.1.7. Strings	15
2.1.8. Functions	15
2.1.9. Array หลายมิติ	16
3. Recursion	17
3.1. แนวคิด	17
3.2. การเขียนโปรแกรม	17
3.3. วิเคราะห์ประสิทธิภาพ	17
3.4. โจทย์ตัวอย่าง - Factorial	17
3.4.1. เนื้อหาโจทย์	17
3.4.2. แนวคิด	17
3.4.3. รหัสเทียม	17
3.5. โจทย์ตัวอย่าง - Fibonacci	18
3.5.1. เนื้อหาโจทย์	18
3.5.2. แนวคิด	18
3.5.3. รหัสเทียม	18

3.6. โจทย์ตัวอย่าง - Permutation	19
3.6.1. เนื้อหาโจทย์	19
3.6.2. แนวคิด	19
3.6.3. รหัสเทียม	19
3.7. ปัญหาที่พบบ่อย	19
3.7.1. Overflow	19
3.8. โจทย์แนะนำ	20
4. Function	21
4.1. แนวคิด	21
4.2. การเขียนโปรแกรม	21
4.2.1. ตัวอย่างฟังก์ชันพร้อมเรียกใช้งาน	21
4.2.2. User-defined functions	22
4.2.3. มุมมองของฟังก์ชัน	22
4.3. โจทย์ตัวอย่าง - ค่าถัดไปของฟังก์ชันเวียนเกิด	23
4.3.1. เนื้อหาโจทย์	23
4.3.2. รหัสเทียม	23
4.4. โจทย์ตัวอย่าง - ระยะห่างระหว่างสองจุด	23
4.4.1. เนื้อหาโจทย์	23
4.4.2. รหัสเทียม	23
4.5. โจทย์ตัวอย่าง - ระยะห่าง Manhattan	23
4.5.1. เนื้อหาโจทย์	23
4.5.2. รหัสเทียม	23
4.6. ปัญหาที่พบบ่อย	23
4.6.1. Scope of variables	23
4.6.2. Return/type	24
4.6.3. Arguments passed by value and by references	24
5. Graph Structure	25
5.1. คำอธิบาย	25
5.2. Example	25
5.3. https://csacademy.com/lesson/introduction_to_graphs/ °	25
5.4. https://csacademy.com/lesson/graph_representation/ °	25
5.5. https://usaco.guide/CPH.pdf#page=119 °	25
6. Linked List	26
6.1. คำอธิบาย	26
6.2. Singly Linked List	26
6.3. ตัวอย่างการ implement singly linked list	26
6.4. Operations	28
6.5. Doubly Linked List	28
7. Dynamic Array	30
7.1. คำอธิบาย	30
7.2. แนวทาง	30
7.3. การเพิ่มข้อมูล	30
7.4. Delete Element	30

7.5. ตัวอย่าง implementation แบบไม่ใช้ vector	30
8. Binary Search Tree	32
8.1. Description	32
8.2. Operations	32
8.3. Example	32
9. Binary Tree	34
9.1. คำอธิบาย	34
9.2. คำศัพท์	34
9.3. เมื่อไรถึงใช้ tree?	34
9.4. Binary Tree	34
9.5. ตัวอย่าง	35
9.6. ใช้ array ในการสร้าง binary tree	36
10. Introduction & Big-O Notation	37
10.1. วันนี้	37
10.2. การเขียนโปรแกรมแข่งขันประกอบด้วย	37
10.3. การออกแบบอัลกอริทึม	37
10.4. การ implement สร้างอัลกอริทึม	37
10.5. การวิเคราะห์อัลกอริทึม	37
10.6. Motivation problem	37
10.7. ประสิทธิภาพของอัลกอริทึม	38
10.8. Time complexity	38
10.9. กฎของการคำนวณ	38
10.10. การวนซ้ำ	38
10.11. Order of magnitude	39
10.12. Phase	39
10.13. กรณีหลายตัวแปร	39
10.14. Recursion	40
10.15. Complexity classes	40
10.16. การประเมินประสิทธิภาพ	41
10.17. ตัวแปร constant	42
10.18. space complexity	42
11. Priority Queue	43
11.1. คำอธิบาย	43
11.2. Operations	43
11.3. Implementation	43
11.4. ตัวอย่าง (STL)	43
12. Set & Map	44
12.1. คำอธิบาย	44
12.2. ข้อแตกต่าง	44
12.3. Set example	44
12.4. Map example	44
13. Heap	46
13.1. คำอธิบาย	46

13.2. คุณสมบัติ	46
13.3. Operations (Min Heap)	46
13.4. ตัวอย่างการ implement	46
14. Review	49
14.1. stack and queue	49
14.2. linked list	49
14.3. dynamic array	49
14.4. binary tree	49
14.5. heap	49
14.6. priority queue	49
14.7. binary search tree	49
14.8. set, map	49
14.9. graph	49
14.10. hash table	50
15. Stack & Queue	51
15.1. Stack	51
15.2. ตัวอย่างการใช้ vector เพื่อ implement stack	51
15.3. Queue	51
15.4. Stack vs Queue	52
15.5. Deque (double-ended queue)	52
15.6. ตัวอย่าง deque	52
16. Hash Table	53
16.1. Idea	53
16.2. Polynomial hashing	53
16.2.1. ตัวอย่าง	53
16.3. ตัวอย่างโปรแกรม	53
16.4. Collision	54
16.4.1. Separate Chaining	54
16.4.2. Open addressing	54
16.4.3. เปรียบเทียบ	55
17. Greedy	56
17.1. หลักการ	56
17.2. ปัญหาการทอนเหรียญ	56
17.2.1. ตัวอย่าง	56
17.2.2. Greedy approach	56
17.3. ปัญหาการจัดตารางเวลา	56
17.3.1. ตัวอย่าง	56
17.3.2. คำอธิบาย	56
17.4. เพิ่มเติม	57
18. Array Manipulation	58
18.1. คำอธิบาย	58
19. Search	59
19.1. หลักการ	59

19.2. Generating Permutations	59
19.3. Graph Traversal	59
19.4. Binary Search Tree	59
19.5. Meet in the middle	59
20. Dynamic Programming	60
20.1. หลักการ	60
20.2. ปัญหาการทอนเหรียญ (ปัญหาเดียวกับ Greedy)	60
20.3. Longest Increasing Subsequence	60
20.4. Review	60
20.4.1. คืออะไร?	60
20.4.2. มีอะไรบ้าง?	60
20.4.3. example	61
20.4.3.1. fibonacci	61
20.4.3.2. LIS	61
20.5. เพิ่มเติม	62
21. Ad-hoc	64
21.1. หลักการ	64
21.2. String	64
22. Divide & Conquer	65
22.1. หลักการ	65
22.1.1. General Pseudocode	65
22.2. Binary Search	65
22.2.1. Pattern	65
22.2.2. STL	65
22.2.3. Binary Search คำตอบ	65
22.3. More on Binary Search	66
22.4. Closest Pair	66
22.5. ปัญหาเลขยกกำลัง	66
22.6. จำนวน Fibonacci	66
22.7. Merge Sort	66
22.8. Square Root Decomposition	67
22.9. เพิ่มเติม	67
23. Graph Algorithm	69
23.1. Day 1	69
23.1.1. Search in Graph	69
23.2. Day 2	69
23.3. Day 3	70
23.4. เพิ่มเติม	70
24. Topological sorting (implementation)	72
25. Paths and circuits	73
25.1. Eulerian	73
25.1.1. Path	73
25.1.2. Circuit	73

25.1.3. Existence	73
25.1.4. Hierholzer's algorithm	73
25.2. Hamiltonian	73
25.2.1. Existence	73
26. Weighted Shortest Path	74
26.0.1. Dijkstra's Algorithm (implementation)	74
26.0.2. Bellman-Ford	75
26.0.3. Floyd-Warshall (implementation)	76
26.0.4. Johnson (exist)	76
26.0.5. A* Search	76
27. Strong connectivity	77
27.0.1. Kosaraju's	77
27.0.2. Challenge	78
28. Recap Graph Day 2	79
28.1. Weighted Shortest Path	79
28.2. Minimum Spanning Tree	79
28.3. Proof	79
29. Minimum Spanning Tree	81
29.1. Kruskal's (implementation)	81
29.2. Prim's	82
30. Graph Applications	83
30.1. Colorings	83
30.2. Connectivity check	83
30.3. Finding cycles	83
30.4. Bipartiteness check	83
31. Tree algorithms	84
31.1. Diameter	84
31.2. Lowest common ancestor (implementation)	84
31.3. Euler Tour Technique	85
31.4. Challenge	85
32. แนวทางการฝึกด้วยตัวเอง	86
32.1. สำหรับก่อน สอน. ค่าย 1	86
32.2. สำหรับช่วงหลัง สอน. ค่าย 1	86
32.3. สำหรับช่วงหลัง สอน. ค่าย 2	86
32.4. สำหรับขั้นต่อไป	86
33. Resources	87
33.1. Recommended Books	87
33.2. Recommended Websites	87
33.3. Random Posts	87

0.1. หลักการ

การเขียนโปรแกรมเชิงแข่งขันแบ่งออกเป็นสองหัวข้อหลักๆ คือ

0.1.1. การออกแบบวิธีการคิด

ส่วนนี้จะเน้นเรื่องทักษะการแก้ปัญหาและพื้นฐานการคิด บางทีต้องมีความคิดสร้างสรรค์ ประกอบกับทฤษฎีต่างๆ โดยทั่วไปแล้ววิธีการแก้ปัญหามักประกอบด้วยเทคนิคที่รู้กันทั่วไปพร้อมกับความเป็นเอกลักษณ์ของปัญหานั้นๆ

0.1.2. การเขียนโปรแกรมตามวิธี

ส่วนนี้จะเน้นว่าเราสามารถถ่ายทอดวิธีการคิดของเราสู่โปรแกรมคอมพิวเตอร์ได้อย่างถูกต้อง ปกติแล้วแนวคิดถูกต้องนั้นไม่เพียงพอในการแข่งขันเขียนโปรแกรมให้ได้ดี ข้อแตกต่างสำคัญกับการเขียนโปรแกรมทั่วไปคือโปรแกรมในการแข่งขันจะสั้น (มากที่สุดหลักไม่กี่ร้อยบรรทัด) และไม่จำเป็นต้องดูแลโปรแกรมหลังจากแข่งเสร็จ

0.2. การบริหารเวลา

เป็นสิ่งสำคัญที่สุดในการแข่งขัน

- ให้คำนึงให้ได้ว่าเราจะใช้เวลาเท่าไรในการทำโจทย์ลักษณะนี้ แล้วจะได้คะแนนเท่าไร แล้วลำดับความสำคัญในการเลือกโจทย์ให้ได้
- พยายามสร้างกฎที่เหมาะสมกับตัวเอง เช่น ถ้าบักเกินสามสิบนาทีแบบไม่มีความคืบหน้า ให้เปลี่ยนไปอ่านข้ออื่นอย่างน้อยห้านาที

0.3. Time Complexity

- ก่อนจะเริ่มลงมือเขียนโปรแกรม เราควรจะต้องรู้ได้ว่าสิ่งที่เราเขียนจะได้กี่คะแนนสำหรับปัญหานั้น
- Big O Notation เป็นสิ่งสำคัญที่ควรจะต้องคาดคะเนให้ได้ว่าเราจะได้คะแนนเท่าไร
- จุดอ้างอิงคร่าวๆ ให้คิดว่า C++ สามารถรันได้ 1,000,000,000 (พันล้าน) คำสั่งต่อหนึ่งวินาที ซึ่งหมายความว่า
 - ▶ $N = 1,000$ สามารถคิดวิธี $O(N^3)$ ได้แบบฉิวเฉียด $O(N^2 \log N)$ ได้อย่างปลอดภัย
 - ▶ $N = 1,000,000$ ควรจะ $O(N \log N)$
 - ▶ $N = 20$ อาจจะเป็น $O(2^N)$
- จำว่า 2^{20} มีค่าประมาณ 1,000,000 (หนึ่งล้าน)

ขนาด N	เวลาที่ใช้
$N \leq 10$	$O(N!)$
$N \leq 20$	$O(2^N)$
$N \leq 500$	$O(N^3)$
$N \leq 5000$	$O(N^2)$
$N \leq 10^6$	$O(N \log N)$ or $O(N)$
N is large	$O(1)$ or $O(\log N)$

สัญลักษณ์	ความหมาย
P	ถูกต้อง
-	Incorrect Output
X	Error
T	Timeout

1. Common STL library

1.1. Max/Min

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    cout << max(2, 4); // 4
    cout << min(2, 4); // 2
    return 0;
}
```

1.2. Sort

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    int a[] = {3, 1, 2};
    cout << a[0] << " " << a[1] << " " << a[2] << "\n"; // 3 1 2
    sort(a, a+3);
    cout << a[0] << " " << a[1] << " " << a[2] << "\n"; // 1 2 3
    return 0;
}
```

1.3. Vector

- เป็น dynamic array (array ที่สามารถเพิ่มลดขนาดได้)
- vector มีการใช้งานโดยใช้ `vector<T> varName;` เช่น `vector<int> A;`

1.3.1. Access

- `A[i]` ได้เหมือน array
- `A.front()` และ `A.back()`

1.3.2. Functions

- `A.size()` เช็คขนาดของ vector
- `A.empty()` เช็คว่างหรือไม่
- `A.push_back(10)` ใส่อะไรเพิ่มสมาชิกต่อท้าย
- `A.pop_back()` ไล่ลบสมาชิกตัวสุดท้าย

1.3.3. Loop

```
for (int i = 0; i < A.size(); i++)
    cout << A[i] << endl;
```

1.3.4. Example

```
#include <bits/stdc++.h>

using namespace std;
```

```

void printVec(vector<int> v) {
    for (int i = 0; i < v.size(); i++) {
        cout << v[i] << " ";
    }
    cout << "\n";
}

int main() {
    vector<int> A;
    A.push_back(2);
    A.push_back(5);
    printVec(A); // OUTPUT: 2 5 \n

    A.push_back(9);
    printVec(A); // OUTPUT: 2 5 9 \n

    cout << A.front() << "\n"; // OUTPUT: 2\n
    cout << A.back() << "\n"; // OUTPUT: 9\n
    cout << A.size() << "\n"; // OUTPUT: 3\n

    A.pop_back();
    A.pop_back();
    printVec(A); // OUTPUT: 2 \n
}

```

1.4. Pair

- เป็น struct สำหรับที่มีสมาชิกสองตัว
- `pair<T1, T2> varName;` เช่น `pair<int, double> A;`

1.4.1. Access

- `A.first` และ `A.second`

1.4.2. Example

```

#include <bits/stdc++.h>

using namespace std;

int main() {
    pair<int, double> A;
    A.first = 4;
    A.second = 7.5;
    cout << A.first << "\n"; // OUTPUT: 4
    cout << A.second << "\n"; // OUTPUT: 7.5
    A = pair<int, double>(6, 2.3);
    cout << A.first << "\n"; // OUTPUT: 6
    cout << A.second << "\n"; // OUTPUT: 2.3
}

```

2. CPP Syntax for Competitive Programming

อ้างอิงจาก [C++ Language Tutorial](https://cplusplus.com/files/tutorial.pdf) และประสบการณ์ส่วนตัว

2.1. Bare Minimum

2.1.1. Code Template

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    return 0;
}
```

- `bits/stdc++.h` เป็น library ที่นิยมใช้กันในการเขียนโปรแกรมแข่งขันเพื่อความสะดวก
- `using namespace std` ใช้เพื่อความสะดวกเวลาใช้คำสั่งต่าง ๆ เช่น
 - ▶ เดิมจะต้องเขียน `std::cout << "Hello World!";` ในการแสดงผลคำว่า `Hello World!`
 - ▶ ถ้ามีบรรทัด `using namespace std` แล้วจะไม่ต้องพิมพ์ `std::` นำหน้า
 - ▶ ก็คือจะสั้นลงเหลือ `cout << "Hello World!";`

2.1.2. Comments

```
// line comment

/*
block
comment
*/
```

2.1.3. Variables Data Types

ในภาษา C++ จำเป็นจะต้องประกาศ “ชนิด” ของตัวแปรเสมอ (ต่างจาก Python)

ชนิดที่พบบ่อยคือ

ชนิด | Size (byte) | — | — | — `bool` | 1 | ค่าความจริง true, false (0-1) `char` | 2 | ตัวอักษร
`int` | 4 | จำนวนเต็ม `long long` | 8 | จำนวนเต็มเหมือนกัน แต่เก็บค่าได้มากกว่า `float` | 4 |
จำนวนจริง `double` | 8 | จำนวนจริงเหมือนกัน แต่เก็บค่าได้มากกว่า

สาเหตุที่มีชนิดเดียวกันที่เก็บค่าได้แตกต่างกันเพราะการจองพื้นที่ memory ขนาดไม่เท่ากัน

2.1.4. Input/Output

ใช้ `cin` ในการรับค่า โดยที่ `cin` จะดูชนิดของตัวแปรให้ตอนที่รับค่า

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    int a;
    cin >> a;
```

```
    return 0;
}
```

หากพิมพ์ 5 หลังจากรันโปรแกรมแล้ว a จะมีค่าเท่ากับ 5 ที่เป็น integer ถ้า

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    char a;
    cin >> a;
    return 0;
}
```

แล้วพิมพ์ 5 หลังจากรันโปรแกรมแล้ว a จะมีค่าเท่ากับ '5' ที่เป็น character ตัวอย่างเช่น

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    char a;
    int b;
    cin >> a >> b;
    int c = a;
    cout << c << " " << b;
    return 0;
}
```

แล้วพิมพ์ 5 5 จะได้ผลลัพธ์เป็น 53 5 เนื่องจาก character '5' มีค่า ascii เป็น 53 (ถูกแปลงค่า) > '\n' เป็น special character ที่ใช้สำหรับขึ้นบรรทัดใหม่ในการแสดงผล เช่น

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    char a;
    int b;
    cin >> a >> b;
    int c = a;
    cout << c << "\n" << b;
    return 0;
}
```

แล้วพิมพ์ 5 5 จะได้ผลลัพธ์เป็น

```
53
5
```

- > สังเกตว่า `cout` ต้องใช้เป็นท่อน ๆ คือส่ง `<<` ได้ทีละตัวแปรหรือทีละ string เท่านั้น
- > เช่นเดียวกัน `cin` ที่ใช้เป็นท่อน ๆ คือรับ `>>` เข้าทีละตัวแปร โดยตัด input ตามประเภทตัวแปร

2.1.5. Control Structures

2.1.5.1. condition

- if-else if-else

```
if (x > 0)
    cout << "x is positive";
else if (x < 0)
    cout << "x is negative";
else
    cout << "x is 0";
```

2.1.5.2. loop

- while

```
int i = 0;
while (i < 10) {
    cout << i << " ";
    i++;
}
```

- for

```
for (int i = 0; i < 10; i++) {
    cout << i << " ";
}
```

2.1.6. Arrays

ใน C++ นั้นตัวแปรทุกชนิดสามารถทำเป็นตัวแปรชุด (Array) ได้ด้วยการใส่วงเล็บ `[]` ด้านหลังชื่อตัวแปร เช่น

จาก `int a` เป็นการประกาศตัวแปรที่เป็นจำนวนเต็มหนึ่งจำนวน

สามารถเป็น `int a[10]` เป็นการประกาศตัวแปรที่เป็นจำนวนเต็ม 10 จำนวน โดยจะเรียกสมาชิกได้ เช่น

`a[0]` เป็นการเรียกสมาชิกตัวแรกใน array นั้น

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    int n;
    int a[20];
    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }
    return 0;
}
```

ตัวอย่างด้านบนจะเป็นการรับค่า `n` เพื่อที่จะรับจำนวนเต็มต่ออีก `n` จำนวน (ใช้ต่อในเงื่อนไขของ for loop)

จากนั้นรับค่าเข้าสู่ `a[i]` ไปเรื่อย ๆ โดย $i = \{0, 1, \dots, n\}$

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    int n;
    int a[20];
    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }
    int sm = 0;
    for (int i = 0; i < n; i++) {
        sm += a[i];
    }
    cout << sm;
    return 0;
}
```

ตัวอย่างด้านบนเป็นการนำค่าที่รับเข้ามาไปใช้หาผลรวมต่อในตัวแปร `sm`

2.1.7. Strings

ใน C++ มี `string` ให้ใช้ใน STL ซึ่งถูกรวมอยู่ใน `bits/stdc++.h` แล้ว

จากที่ภาษา C ธรรมดาจะต้องใช้ เป็น array of characters จึงสะดวกขึ้นเยอะ

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    string a;
    cin >> a;
    cout << a;
    return 0;
}
```

ตัวอย่างด้านบนเป็นการรับค่า string แล้วส่งออกเลย

2.1.8. Functions

```
#include <bits/stdc++.h>

using namespace std;

int addint(int a, int b) {
    return a + b;
}

int main() {
    cout << addint(1, 4);
}
```

```
    return 0;
}
```

ตัวอย่างด้านบนเป็นการสร้างฟังก์ชัน $f(a, b) = a + b$ โดยให้ชื่อว่า `addint` พร้อมตัวอย่างการเรียกใช้

2.1.9. Array หลายมิติ

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int main() {
    int a[5][5];
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> a[i][j];
        }
    }
    return 0;
}
```


3. Recursion

3.1. แนวคิด

- Recursion คือกระบวนการเรียกตัวเอง
- ตามปกติแล้วแบ่งกรณีเป็น
 - base case
 - recursion case

โดยที่จะย่อยปัญหาให้เล็กลงจนกระทั่งแก้ได้ด้วย base case

3.2. การเขียนโปรแกรม

จะเขียนกระบวนการนั้นด้วยฟังก์ชัน และการเรียกตัวเองจะเป็นการเรียกฟังก์ชันที่เขียนขึ้นมาภายในฟังก์ชันนั้น

ตัวอย่างการเขียน recursive function ในภาษา C++ โดยยังไม่คำนึงถึงการนำไปใช้แก้โจทย์ปัญหา

```
void recursive_function() {  
    if (base condition)  
        return; // base case  
    else  
        recursive_function(); // recursion case  
}
```

3.3. วิเคราะห์ประสิทธิภาพ

จะใช้วิธีการนับจำนวนครั้งที่จะมีการเรียกฟังก์ชันเกิดขึ้นตามพารามิเตอร์ของฟังก์ชันนั้น

3.4. โจทย์ตัวอย่าง - Factorial

3.4.1. เนื้อหาโจทย์

จงหาค่าของ $n!$ โดย $n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$

3.4.2. แนวคิด

โจทย์นี้สามารถใช้โครงสร้าง loop ในการแก้ปัญหาได้ แต่นำมาเป็นตัวอย่างเริ่มต้นในการใช้โครงสร้างของ recursion ในการแก้ปัญหา

โดยมองค่าของ $n!$ เป็น recursive function ดังนี้

$$f(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ n \times f(n - 1) & \text{else} \end{cases} \quad (1)$$

3.4.3. รหัสเทียม

```
#include <bits/stdc++.h>  
  
using namespace std;  
  
int factorial(int n) {  
    if (n ≤ 1) { // base case  
        return n;  
    } else {  
        return n * factorial(n - 1);  
    }  
}
```

```

    }
}

int main() {
    cout << factorial(4);
    return 0;
}

```

จะได้ output

24

3.5. โจทย์ตัวอย่าง - Fibonacci

3.5.1. เนื้อหาโจทย์

จงหาค่าของเลขฟีโบนัชชีลำดับที่ n โดยที่ เลขฟีโบนัชชีลำดับที่ n จะมีค่าเท่ากับผลบวกของเลขฟีโบนัชชีลำดับที่ $n - 1$ และ $n - 2$

3.5.2. แนวคิด

โจทย์นี้สามารถใช้โครงสร้าง loop ในการแก้ปัญหาได้เช่นกัน แต่นำมาเป็นอีกตัวอย่างหนึ่งในการใช้โครงสร้างของ recursion ในการแก้ปัญหา

โดยมองค่าของเลขฟีโบนัชชีลำดับที่ n เป็น recursive function ดังนี้ $f(n) = \begin{cases} n & \text{if } n \leq 1, \\ f(n-1) + f(n-2) & \text{else} \end{cases}$

3.5.3. รหัสเทียม

```

#include <bits/stdc++.h>

using namespace std;

int fibo(int n) {
    if (n ≤ 1) { // base case
        return 1;
    } else {
        return fibo(n - 1) + fibo(n - 2);
    }
}

int main() {
    cout << fibo(4);
    return 0;
}

```

จะได้ output

3

3.6. โจทย์ตัวอย่าง - Permutation

3.6.1. เนื้อหาโจทย์

จงหาลำดับที่มีสมาชิก n ตัว โดยที่แต่ละสมาชิกมีค่าน้อยกว่า r ทั้งหมดที่เป็นไปได้ (จำเป็นต้องใช้ array)

3.6.2. แนวคิด

เป็นการค้นหาวีธีที่เป็นไปได้ (complete search) ซึ่งจะนำ recursive function เข้ามาใช้ โดยเริ่มต้นด้วยการสร้างลำดับที่ยังไม่มีสมาชิกใด ๆ และค่อย ๆ เติมสมาชิกเข้าไปในลำดับนั้น

base case จะเป็นเงื่อนไขเมื่อลำดับมีสมาชิกครบ n

recursive case จะเป็นการเติมสมาชิกลำดับถัดไปด้วยทุกค่าที่เป็นไปได้

3.6.3. รหัสเทียม

```
#include <bits/stdc++.h>

using namespace std;

int a[10];

void perm(int d, int r, int n) {
    if (d == n) { // base case
        for (int i = 0; i < n; i++) {
            cout << a[i] << " ";
        }
        cout << "\n";
        return;
    }
    for (int i = 0; i < r; i++) {
        a[d] = i;
        perm(d + 1, r, n); // recursive case
    }
}

int main() { perm(0, 3, 3); }
```

จะได้ output

```
0 0 0
0 0 1
...
2 2 2
```

3.7. ปัญหาที่พบบ่อย

3.7.1. Overflow

กรณีไม่มี base case หรือ base case ไม่ครอบคลุม จะทำให้เกิดการ overflow ได้

```
#include <bits/stdc++.h>

using namespace std;
```

```
int factorial(int n) { return n * factorial(n - 1); }

int main() {
    cout << factorial(4);
    return 0;
}
```

จะเกิด error เช่น:

"/a.out" terminated by signal SIGSEGV (Address boundary error)

3.8. โจทย์แนะนำ

- หาลำดับ permutation โดยมีเงื่อนไขที่แตกต่างกัน
 - จำนวน 5 หลัก และแต่ละหลักมีค่าไม่เกิน 6
 - ตัวเลขก่อนหน้าต้องน้อยกว่าหรือเท่ากับตัวถัดไป
 - ตัวเลขห้ามซ้ำ
- กลับข้อความ (reverse string) ห้ามใช้ loop, array หรือ string
 - กลับทั้งข้อความ เช่น: "iamastring" เป็น "gnirtsamai"
 - กลับเฉพาะภายในคำที่คั่นด้วยเว้นวรรค เช่น: "i am a string" เป็น "i ma a gnirts"
- Programming.in.th
 - 0019 Perket
 - 0039 Food
- cses.fi
 - 2205 Gray Code
 - 2165 Tower of Hanoi
 - 1622 Creating Strings
 - 1623 Apple Division

4. Function

- ตัวอย่างฟังก์ชันที่มีใน `bits/stdc++.h`
- วิธีการเขียน user-defined functions
- เทียบมุมมองของฟังก์ชันทางคณิตศาสตร์และทางคอมพิวเตอร์

4.1. แนวคิด

อ่าน official tutorial เรื่อง functions ที่ [cplusplus.com](https://cplusplus.com/doc/tutorial/functions/)

การเขียนฟังก์ชันในโปรแกรมเป็นการรวมกลุ่มคำสั่งเพื่อทำงานตามจุดประสงค์ โดยสามารถเรียกใช้งานภายหลังในโปรแกรมนั้นได้

4.2. การเขียนโปรแกรม

4.2.1. ตัวอย่างฟังก์ชันพร้อมเรียกใช้งาน

ใน C++ Standard Template Library (STL) จะมีฟังก์ชันพร้อมเรียกใช้งานให้ใช้ ซึ่งเมื่อ

```
#include <bits/stdc++.h>
```

แล้ว ในการเขียนโปรแกรมเชิงแข่งขันจะทำการ include STL ที่จำเป็นทั้งหมดมาให้ ซึ่งตัวอย่างฟังก์ชันที่ใช้งานบ่อยครั้งมีดังนี้

- `abs`, `pow`, `sqrt`, `log2`, `exp` จาก `cmath`
- `swap`, `min`, `max`, `__gcd`, `next_permutation` จาก `algorithm`

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int main() {
    cout << "cmath examples\n";
    cout << "abs(-5): " << abs(-5) << "\n";
    cout << "pow(2, 3): " << pow(2, 3) << "\n";
    cout << "sqrt(64): " << sqrt(64) << "\n";
    cout << "log2(64): " << log2(64) << "\n";
    cout << "exp(3): " << exp(3) << "\n";
    cout << "\n";

    cout << "algorithm examples\n";
    int a = 4, b = 7;
    cout << "a b: " << a << " " << b << "\n";
    swap(a, b);
    cout << "a b: " << a << " " << b << "\n";
    cout << "min(a, b): " << min(a, b) << "\n";
    cout << "max(a, b): " << max(a, b) << "\n";
    cout << "__gcd(a, b): " << __gcd(a, b) << "\n";
}
```

Output:

```
cmath examples
abs(-5): 5
pow(2, 3): 8
sqrt(64): 8
```

```
log2(64): 6
exp(3): 20.0855
```

```
algorithm examples
a b: 4 7
a b: 7 4
min(a, b): 4
max(a, b): 7
__gcd(a, b): 1
```

4.2.2. User-defined functions

เราสามารถเขียนฟังก์ชันด้วยตนเองได้ โดยการระบุ: • ชนิด (type) ของค่าที่จะคืนค่า • ชื่อ (name) ที่ใช้เรียกฟังก์ชันนั้น • พารามิเตอร์ (parameters) ที่ไว้รับค่าเข้าสู่ฟังก์ชัน • ชุดคำสั่ง (statements) ที่ระบุวิธีการทำงานของฟังก์ชันนั้น

ในรูปแบบ:

```
type name ( param1, param2, ... ) { statements }
```

ตัวอย่างการเขียนฟังก์ชัน:

```
#include <bits/stdc++.h>
using namespace std;

int addition(int a, int b) {
    int r;
    r = a + b;
    return r;
}

int main() {
    int z;
    z = addition(5, 3);
    cout << "The result is " << z;
}
```

Output:

The result is 8

4.2.3. มุมมองของฟังก์ชัน

ทางคณิตศาสตร์จะเขียนฟังก์ชันอยู่ในรูป $y = f(x)$ ซึ่งเมื่อเทียบกับทางคอมพิวเตอร์แล้วจะเทียบเท่าเป็นฟังก์ชันที่มี: • ไม่ได้ระบุชนิดค่าที่จะคืนออกมา • ชื่อ f • รับพารามิเตอร์ x

ตัวอย่างฟังก์ชันเอกลักษณ์:

$f(x) = x$

ในภาษา C++:

```
int f(int x) {
    return x;
}
```

4.3. โจทย์ตัวอย่าง - ค่าถัดไปของฟังก์ชันเวียนเกิด

4.3.1. เนื้อหาโจทย์

จงเขียนฟังก์ชัน `next_a(int p, int pn)` หาค่า:

$$a_n = a_{\{n-1\}} + n \times 2$$

เมื่อทราบค่า $a_{\{n-1\}}$

4.3.2. รหัสเทียม

```
int next_a(int p, int pn) {  
    return p + (pn + 1) * 2;  
}
```

4.4. โจทย์ตัวอย่าง - ระยะห่างระหว่างสองจุด

4.4.1. เนื้อหาโจทย์

กำหนดให้ $A(x_1, y_1)$ และ $B(x_2, y_2)$ เป็นจุดสองจุดบนระนาบ

4.4.2. รหัสเทียม

```
double distance(double x1, double y1, double x2, double y2) {  
    return sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2));  
}
```

4.5. โจทย์ตัวอย่าง - ระยะห่าง Manhattan

4.5.1. เนื้อหาโจทย์

จงเขียนฟังก์ชัน `manhattan(x1, y1, x2, y2)` เพื่อหาระยะ Manhattan ระหว่าง A และ B

4.5.2. รหัสเทียม

```
double manhattan(double x1, double y1, double x2, double y2) {  
    return abs(x1 - x2) + abs(y1 - y2);  
}
```

4.6. ปัญหาที่พบบ่อย

4.6.1. Scope of variables

แต่ละฟังก์ชันมี scope ของตัวเอง ตัวแปรแบ่งเป็น:

- Global variables
- Local variables

ตัวอย่าง:

```
#include <bits/stdc++.h>  
using namespace std;  
  
int g;  
  
int addition(int a, int b) {  
    int r;  
    g = b;  
}
```

```

    r = a + b;
    return r;
}

int main() {
    int z;
    z = addition(5, 3);
    cout << "z is " << z << "\n";
    cout << "g is " << g;
}

```

Output:

```

z is 8
g is 3

```

Errors when accessing out-of-scope variables:

```

cout << "z is " << z << "\n";

```

4.6.2. Return/type

ลืม return จะเกิด warning

```

#include <bits/stdc++.h>
using namespace std;

int addition(int a, int b) {
    int r;
    r = a + b;
}

```

Output:

```

warning: no return statement in function returning non-void

```

4.6.3. Arguments passed by value and by references

เพื่อให้เปลี่ยนค่าพารามิเตอร์ใช้ & ข้างหน้า

ตัวอย่าง:

```

void swap(int &a, int &b) {
    int c = a;
    a = b;
    b = c;
}

```

```

in main: 0 1
in swap: 1 0
in main: 1 0

```


5. Graph Structure

5.1. คำอธิบาย

- กราฟเป็นโครงสร้างข้อมูลไม่เชิงเส้น ประกอบด้วย node (vertex) และ edge
- มีนิยามดังนี้

> A Graph consists of a finite set of vertices (or nodes) and set of Edges which connect a pair of nodes

5.2. Example

![<https://www.geeksforgeeks.org/wp-content/uploads/undirectedgraph.png>]

(<https://www.geeksforgeeks.org/wp-content/uploads/undirectedgraph.png>)

- ในกราฟด้านบนประกอบด้วย set of vertices $V = \{0, 1, 2, 3, 4\}$ และ set of edges $E = \{01, 12, 23, 34, 04, 14, 13\}$.
- กราฟสามารถใช้ในการแก้ปัญหาลงได้หลายปัญหา
- กราฟสามารถใช้แสดงแทนเครือข่าย
- เช่น เส้นทางในเมือง หรือเครือข่ายโทรศัพท์ หรือวงจรไฟฟ้า เป็นต้น
- กราฟถูกใช้ใน social network เช่น LinkedIn และ Facebook
- เช่น ใน Facebook บุคคลเป็น node ที่เก็บข้อมูลไอดี ชื่อ เพศ ต่าง ๆ และเชื่อมกันด้วยเส้นเชื่อมประเภทต่าง ๆ เช่น การเป็นเพื่อน

5.3. https://csacademy.com/lesson/introduction_to_graphs/

- Undirected graph
- Directed graph

5.4. https://csacademy.com/lesson/graph_representation/

- adjacency matrix
- adjacency list

5.5. <https://usaco.guide/CPH.pdf#page=119>

- graph traversal

6.1. คำอธิบาย

-

[<https://upload.wikimedia.org/wikipedia/commons/6/6d/Singly-linked-list.svg>]

(<https://upload.wikimedia.org/wikipedia/commons/6/6d/Singly-linked-list.svg>)

- ## 6.2. Singly Linked List

- ### 6.3. ตัวอย่างการ implement singly linked list

- ```
// A simple CPP program to introduce
// a linked list
#include <bits/stdc++.h>
using namespace std;

// A linked list node
struct Node {
 int data;
 struct Node *next;
};

int main() {
 // Program to create a simple linked
 // list with 3 nodes
 Node *head = NULL;
 Node *second = NULL;
 Node *third = NULL;

 // allocate 3 nodes in the heap
 head = new Node();
 second = new Node();
 third = new Node();

 /* Three blocks have been allocated dynamically.
 We have pointers to these three blocks as head,
 second and third
 head second third
 | | |
```



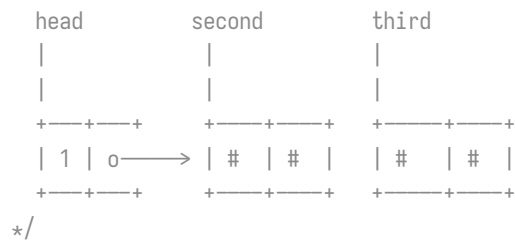
# represents any random value.  
Data is random because we haven't assigned anything yet \*/

```

head->data = 1; // assign data in first node
head->next = second; // Link first node with
// the second node

```

/\* data has been assigned to the data part of first block (block pointed by the head). And next pointer of the first block points to second. So they both are linked.



```

// assign data to second node
second->data = 2;

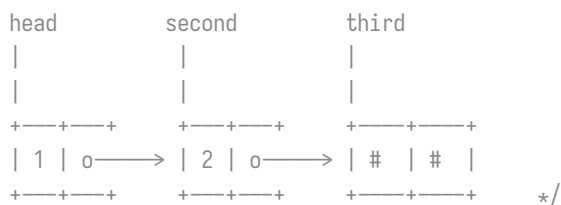
```

```

// Link second node with the third node
second->next = third;

```

/\* data has been assigned to the data part of the second block (block pointed by second). And next pointer of the second block points to the third block. So all three blocks are linked.



```

third->data = 3; // assign data to third node
third->next = NULL;

```

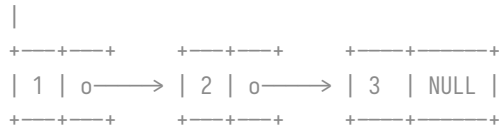
/\* data has been assigned to the data part of the third block (block pointed by third). And next pointer of the third block is made NULL to indicate that the linked list is terminated here.

We have the linked list ready.

```

head
|

```



Note that only the head is sufficient to represent the whole list. We can traverse the complete list by following the next pointers. \*/

```
while (head != NULL) {
 printf("%d ", head->data);
 head = head->next;
}

// This code is contributed by rathbhupendra
}
```

## 6.4. Operations

- insert การเพิ่มข้อมูลใน linked list โดย
  - สร้าง node ใหม่ชื่อว่า newNode (37)
  - นำ pointer ของโหนดตำแหน่งก่อนหน้าที่ต้องการจะแทรก (12) ชี้ไปที่ newNode (37)
  - นำ pointer ของ newNode ไปชี้ที่ pointer ของโหนดตำแหน่งก่อนหน้าเคยชี้ (99)

![[https://upload.wikimedia.org/wikipedia/commons/thumb/4/4b/CPT-LinkedLists-addingnode.svg/474px-CPT-LinkedLists-addingnode.svg.png|https://upload.wikimedia.org/wikipedia/commons/thumb/4/4b/CPT-LinkedLists-addingnode.svg/474px-CPT-LinkedLists-addingnode.svg.png]]

- delete การลบข้อมูลใน linked list โดย
  - นำ pointer ของโหนดตำแหน่งก่อนหน้าที่ต้องการจะลบ (12) ชี้ข้ามไปที่ node ในตำแหน่งถัดไป (37)

![[https://upload.wikimedia.org/wikipedia/commons/thumb/d/d4/CPT-LinkedLists-deletingnode.svg/380px-CPT-LinkedLists-deletingnode.svg.png|https://upload.wikimedia.org/wikipedia/commons/thumb/d/d4/CPT-LinkedLists-deletingnode.svg/380px-CPT-LinkedLists-deletingnode.svg.png]]

## 6.5. Doubly Linked List

- เป็น linked list ที่มีความซับซ้อนเพิ่มขึ้นนิดหนึ่งโดย
- node จะเก็บ pointers สองตำแหน่งสำหรับโหนดก่อนหน้า **prev** และโหนดถัดไป **next**

![[https://upload.wikimedia.org/wikipedia/commons/thumb/5/5e/Doubly-linked-list.svg/610px-Doubly-linked-list.svg.png|https://upload.wikimedia.org/wikipedia/commons/thumb/5/5e/Doubly-linked-list.svg/610px-Doubly-linked-list.svg.png]]

```
// linked.c
#include <bits/stdc++.h>
using namespace std;

// what a node look like internally
struct Node {
```

```

int idx;
struct Node *r_next;
struct Node *r_prev;
};

int main() {
 // define 3 nodes
 struct Node n0, n1, n2;

 // initialize the nodes
 n0.idx = 0; // the first node has an index of 0
 n0.r_next = &n1; // with a reference to the second node
 n0.r_prev = NULL; // and a reference to NULL b/c there's no previous node

 n1.idx = 1;
 n1.r_next = &n2;
 n1.r_prev = &n0;

 n2.idx = 2;
 n2.r_next = NULL;
 n2.r_prev = &n1;

 // create a new reference to n0 called r_first_element
 struct Node *r_first_element = &n0;
 printf("The first node lives @ %p\n", r_first_element);
 printf("The first node has idx: %d\n", r_first_element->idx);

 // print n1 info
 printf("The second node lives @ %p\n", r_first_element->r_next);
 printf("The second node has idx: %d\n", r_first_element->r_next->idx);

 // print n2 info
 printf("The third node lives @ %p\n", r_first_element->r_next->r_next);
 printf("The third node has idx: %d\n", r_first_element->r_next->r_next->idx);
}

```

## 7. Dynamic Array

### 7.1. คำอธิบาย

- dynamic array เป็น array ที่สามารถยืดหดขนาดได้
- ใน STL จะมี **vector** เป็น dynamic array ให้ใช้

### 7.2. แนวทาง

- เมื่อต้องการเพิ่มข้อมูลลงในอาเรย์ แต่อาเรย์นั้นเต็มแล้ว ให้สร้างฟังก์ชันโดย
- ฟังก์ชันนั้นสร้างอาเรย์ขนาดสองเท่าจากเดิม และคัดลอกข้อมูลจากอาเรย์เดิมมาใส่

### 7.3. การเพิ่มข้อมูล

- หากมีการขยายขนาดอาเรย์และคัดลอกข้อมูลจะทำให้ใช้เวลา  $O(n)$
- แต่กรณีนี้เกิดขึ้นเมื่ออาเรย์เต็มเท่านั้น
- ดังนั้นในกรณีส่วนมากยังใช้เวลาเป็น  $O(1)$  อยู่ดี
- ด้วย amortized analysis สามารถถ่วงเฉลี่ยการเพิ่มข้อมูลเป็น  $O(1)$  ได้

![https://media.geeksforgeeks.org/wp-content/uploads/dynamicarray.png°](https://media.geeksforgeeks.org/wp-content/uploads/dynamicarray.png°)

### 7.4. Delete Element

- การลบข้อมูลสามารถทำได้ตามปกติเหมือนอาเรย์ทั่วไป
- ในทางทฤษฎี สามารถลดขนาดอาเรย์ลงได้ แต่ไม่จำเป็นใน competitive programming

![https://media.geeksforgeeks.org/wp-content/uploads/DeleteArray.png°](https://media.geeksforgeeks.org/wp-content/uploads/DeleteArray.png°)

### 7.5. ตัวอย่าง implementation แบบไม่ใช้ vector

```
#include <bits/stdc++.h>

using namespace std;

int cnt = 0;
int sz = 1;
int *A = new int[sz];

void add(int x) {
 if (cnt == sz) {
 sz *= 2;
 int *tmp = new int[sz];
 for (int i = 0; i < cnt; i++) {
 tmp[i] = A[i];
 }
 free(A);
 A = tmp;
 }
 A[cnt++] = x;
}

void remove() { cnt--; }
```

```

void removeAt(int j) {
 for (int i = j; i < cnt; i++) {
 A[i] = A[i + 1];
 }
 cnt--;
}

void print() {
 printf("sz = %d, cnt = %d\\n", sz, cnt);
 for (int i = 0; i < cnt; i++) {
 printf("%d ", A[i]);
 }
 printf("\\n");
}

int main() {
 add(1);
 print();
 add(2);
 print();
 add(3);
 print();
 add(4);
 add(5);
 print();
 add(6);
 print();
 remove();
 print();
 removeAt(3);
 print();
}

```

## 8. Binary Search Tree

### 8.1. Description

- Binary Search Tree เป็น binary tree ที่มีคุณสมบัติเพิ่มเติม
  - ทุกโหนดใน subtree ด้านซ้ายจะต้องมีค่าน้อยกว่า root
  - ทุกโหนดใน subtree ด้านขวาจะต้องมีค่ามากกว่า root
  - subtree ด้านซ้ายและด้านขวาจะต้องเป็น binary search tree

![(https://media.geeksforgeeks.org/wp-content/uploads/BSTSearch.png)](https://media.geeksforgeeks.org/wp-content/uploads/BSTSearch.png)

### 8.2. Operations

- Search
- Insertion
- Traversal
  - Pre-order
  - In-order
  - Post-order

### 8.3. Example

```
#include <stdio>
#include <string>
int tree[1000];

int insert(int n, int p) {
 if (tree[p] == -1)
 return p;
 if (n < tree[p])
 return insert(n, p * 2 + 1);
 else
 return insert(n, p * 2 + 2);
}

void pre(int p) {
 if (tree[p] == -1)
 return;
 printf("%d\n", tree[p]);
 pre(p * 2 + 1);
 pre(p * 2 + 2);
}

void in(int p) {
 if (tree[p] == -1)
 return;
 in(p * 2 + 1);
 printf("%d\n", tree[p]);
 in(p * 2 + 2);
}

void pos(int p) {
 if (tree[p] == -1)
```



```

 return;
 pos(p * 2 + 1);
 pos(p * 2 + 2);
 printf("%d\n", tree[p]);
}

int main() {
 char a[5];
 scanf("%s", a);
 int x;
 scanf("%d", &x);
 int i;
 for (i = 0; i < 1000; i++)
 tree[i] = -1;
 while (x--) {
 int n;
 scanf("%d", &n);
 if (tree[0] == -1)
 tree[0] = n;
 else
 tree[insert(n, 0)] = n;
 }

 if (!strcmp(a, "PRE"))
 pre(0);
 if (!strcmp(a, "POS"))
 pos(0);
 if (!strcmp(a, "INF"))
 in(0);
 if (!strcmp(a, "BRF"))
 for (i = 0; i < 1000; i++)
 if (tree[i] != -1)
 printf("%d\n", tree[i]);
 return 0;
}

```

## 9. Binary Tree

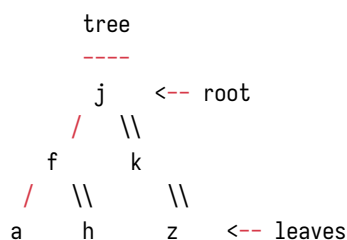
### 9.1. คำอธิบาย

- tree เป็นโครงสร้างข้อมูลแบบมีชั้น ซึ่งแตกต่างจาก array, linked list, stack และ queue ที่เป็นโครงสร้างเชิงเส้น
- ตัวอย่าง tree เบื้องต้นคือ binary tree โดยแต่ละ node สามารถมีลูกได้มากที่สุดไม่เกิน 2 ลูก ดังรูป
- ซึ่งเราสามารถเรียกลูกว่า left และ right child ได้

![https://media.geeksforgeeks.org/wp-content/cdn-uploads/binary-tree-to-DLL.png°]  
(https://media.geeksforgeeks.org/wp-content/cdn-uploads/binary-tree-to-DLL.png°)

### 9.2. คำศัพท์

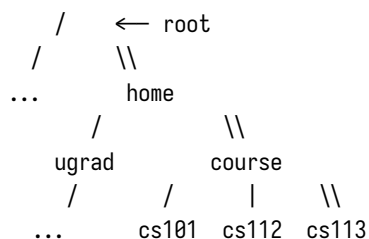
- node ด้านบนสุดจะถูกเรียกว่า **root**
- ลูกด้านล่างที่อยู่ติดกับ node โดยตรงจะเรียกว่า **children**
- node ด้านบนจะเรียกว่า **parent**
- เช่น a เป็น child ของ f และ f เป็น parent ของ a
- และ node ที่ไม่มีลูกเลยจะเรียกว่า **leaves**



### 9.3. เมื่อไรถึงใช้ tree?

- หากต้องการเก็บข้อมูลที่มีลำดับชั้น เช่น ระบบไฟล์ในคอมพิวเตอร์

file system



- tree (ประกอบกับ ordering บางอย่าง e.g., BST) สามารถเข้าถึงข้อมูลได้เร็วกว่า linear data structure
- tree สามารถเพิ่มลบข้อมูลได้สะดวกกว่าในบางกรณี
- tree สามารถ implement ในรูปแบบคล้ายกับ linked list ได้

### 9.4. Binary Tree

- คือ tree ที่แต่ละ node มีลูกไม่เกินสองลูก คือ left และ right

## 9.5. ตัวอย่าง

- Binary Tree node มีส่วนประกอบต่อไปนี้
  - ข้อมูล
  - Pointer to left child
  - Pointer to right child

```
#include <bits/stdc++.h>
using namespace std;
```

```
struct Node {
 int data;
 struct Node *left;
 struct Node *right;

 // val is the key or the value that
 // has to be added to the data part
 Node(int val) {
 data = val;

 // Left and right child for node
 // will be initialized to null
 left = NULL;
 right = NULL;
 }
};
```

```
int main() {
 /*create root*/
 struct Node *root = new Node(1);
 /* following is the tree after above statement
```

```
 1
 / \
 NULL NULL
*/
```

```
 root->left = new Node(2);
 root->right = new Node(3);
 /* 2 and 3 become left and right children of 1
```

```
 1
 / \
 2 3
 / \ / \
NULL NULL NULL NULL
*/
```

```
 root->left->left = new Node(4);
 /* 4 becomes left child of 2
```

```
 1
 / \
 2 3
 / \ / \
4 NULL NULL NULL
 / \
NULL NULL
```

```
*/

return 0;
}
```

## 9.6. ใช้ array ในการสร้าง binary tree

- ขนาดเรารู้ขนาดของ binary tree แล้วเราสามารถใช้ array ได้
- Consider node  $A[i]$  has
  - $A[i*2]$  as left child
  - $A[i*2+1]$  as right child

## 10. Introduction & Big-O Notation

### 10.1. วันนี้

- introduction
- time complexity
- space complexity

### 10.2. การเขียนโปรแกรมแข่งขันประกอบด้วย

- การออกแบบอัลกอริทึม
- การ implement สร้างอัลกอริทึม

### 10.3. การออกแบบอัลกอริทึม

- ใช้ทักษะ problem solving
- และ mathematical thinking
- โดยจำเป็นต้องวิเคราะห์ปัญหาและแก้ปัญหาย่างสร้างสรรค์
- อัลกอริทึมที่ใช้แก้ปัญหามันจะต้องถูกต้องแล้วมีประสิทธิภาพ
- หลักสำคัญของโจทย์ปัญหาส่วนใหญ่จะเป็นการคิดสร้างสรรค์อัลกอริทึมที่มีประสิทธิภาพ
- ความรู้ทางทฤษฎีของอัลกอริทึมเป็นสิ่งสำคัญต่อนักเขียนโปรแกรมแข่งขัน
- โดยทั่วไปแล้ว เฉลย (solution) ของปัญหาหนึ่งนั้นจะเป็นการใช้เทคนิคที่แพร่หลายร่วมกับ insight ใหม่ตามโจทย์
- เทคนิคในการเขียนโปรแกรมแข่งขันยังเป็นพื้นฐานในการวิจัยสายอัลกอริทึมอีกด้วย

### 10.4. การ implement สร้างอัลกอริทึม

- ใช้ทักษะการเขียนโปรแกรมที่ดี
- ในการเขียนโปรแกรมแข่งขันนั้น เฉลยจะถูกตรวจสอบอัลกอริทึมด้วยชุดทดสอบ
- ดังนั้น การที่โอเคของอัลกอริทึมถูกต้องเท่านั้นยังไม่เพียงพอ แต่การ implement สร้างอัลกอริทึมจะต้องถูกต้องด้วยเช่นกัน
- style ในการเขียนโค้ดในการแข่งขันจะตรงไปตรงมาและกระชับรัด
- โปรแกรมควรถูกเขียนอย่างรวดเร็ว เพราะเวลาจะมีอยู่อย่างจำกัด

### 10.5. การวิเคราะห์อัลกอริทึม

- ปัญหาที่กำหนดให้สามารถแก้ปัญหได้ด้วยอัลกอริทึมหลายแบบในสภาพเงื่อนไขเดียวกัน
- จำเป็นต้องเรียนรู้วิธีการเปรียบเทียบประสิทธิภาพการทำงานของแต่ละอัลกอริทึม

เพื่อพิจารณาเลือกใช้ได้อย่างถูกวิธี

### 10.6. Motivation problem

- เราจะรู้ได้อย่างไรว่าโค้ดไหนมีประสิทธิภาพที่ดีกว่ากัน ระหว่าง

```
for (int i = 1; i ≤ n; i++) {
 for (int j = i + 1; j ≤ n; j++) {
 // code
 }
}
```

- และ

```

for (int i = 1; i ≤ n; i++) {
 for (int j = 1; j ≤ n; j += i) {
 // code
 }
}

```

## 10.7. ประสิทธิภาพของอัลกอริทึม

- สามารถแบ่งได้เป็นสองประเภท คือ
  - การใช้เนื้อที่ในหน่วยความจำ หรือความต้องการในการใช้เนื้อที่ของความจำในการเก็บข้อมูล
  - ประสิทธิภาพของเวลาในการทำงาน หรือระยะเวลาที่ใช้ในการประมวลผลข้อมูล
- โดยทั่วไปแล้วประสิทธิภาพทั้งสองประเภทจะพัฒนาไปพร้อมกันได้ยาก

และส่วนมากจะให้ความสำคัญกับการใช้เวลาอย่างมีประสิทธิภาพมากกว่าเนื้อที่

- **Big O Notation** หรืออันดับขนาด (order of magnitude) เป็นเครื่องมือหลักในการวิเคราะห์อัลกอริทึม

## 10.8. Time complexity

- ประสิทธิภาพของอัลกอริทึมเป็นสิ่งสำคัญในการเขียนโปรแกรมเชิงแข่งขัน
- โดยทั่วไปนั้นจะง่ายที่จะออกแบบอัลกอริทึมที่แก้ปัญหาได้ช้า แต่ความท้าทายจะอยู่ที่อยากออกแบบอัลกอริทึมที่รวดเร็ว
- หากอัลกอริทึมนั้นช้าเกินไป จะได้รับคะแนนบางส่วนหรือไม่ได้เลย
- การคาดการณ์เวลาที่อัลกอริทึมใช้สำหรับ input ต่าง ๆ
- หลักการคือการแสดงประสิทธิภาพในรูปแบบของฟังก์ชันโดยมีพารามิเตอร์เป็นขนาดของ input
- ด้วยการคำนวณ time complexity เราสามารถตรวจสอบความเร็วของอัลกอริทึมได้โดยไม่ต้อง implement ขึ้นมาก่อน

## 10.9. กฎของการคำนวณ

- time complexity ของอัลกอริทึมหนึ่งจะถูกแทนด้วยสัญลักษณ์  $O(c...)$  โดยจุดสามจุดแสดงถึงฟังก์ชันใด ๆ
- โดยทั่วไปแล้ว ตัวแปร  $n$  แสดงถึงขนาดของ input
- ตัวอย่างเช่น หาก input เป็นอาร์เรย์ของตัวเลข  $n$  จะเป็นขนาดของอาร์เรย์นั้น และหาก input เป็น string  $n$  จะเป็นความยาวของ string นั้น

## 10.10. การวนซ้ำ

- เหตุผลที่พบบ่อยจากการที่อัลกอริทึมช้าเกินไปเกิดจากการใช้การวนซ้ำจำนวนมากสำหรับ input
- ยิ่งวนซ้ำหลายชั้น อัลกอริทึมจะยิ่งช้า
- หากมีการวนซ้ำ  $k$  ชั้น time complexity จะเป็น  $O(n^k)$

ตัวอย่างเช่น time complexity ของโค้ดด้านล่างจะเป็น  $O(n)$

```

for (int i = 1; i ≤ n; i++) {
 // code
}

```

และ time complexity ของโค้ดด้านล่างนี้  $O(n^2)$

```

for (int i = 1; i ≤ n; i++) { // ชั้นที่ 1
 for (int j = 1; j ≤ n; j++) { // ชั้นที่ 2

```

```

 // code
}
}

```

### 10.11. Order of magnitude

- time complexity จะไม่สนใจจำนวนรอบเป๊ะ ๆ ภายใน loop แต่จะสนใจเฉพาะ order of magnitude
- ในตัวอย่างต่อไปนี้ จำนวนรอบในการวนเป็น  $3n$ ,  $n + 5$  และ  $\lceil \frac{n}{2} \rceil$  ครั้ง แต่ time complexity นั้นเท่ากันคือ  $O(n)$

```

for (int i = 1; i ≤ 3*n; i++) {
 // code
}

for (int i = 1; i ≤ n+5; i++) {
 // code
}

for (int i = 1; i ≤ n; i += 2) {
 // code
}

```

ในอีกตัวอย่างหนึ่ง time complexity ของโค้ดต่อไปนี้เป็น  $O(n^2)$

```

for (int i = 1; i ≤ n; i++) {
 for (int j = i+1; j ≤ n; j++) {
 // code
 }
}

```

### 10.12. Phase

- ถ้าอัลกอริทึมประกอบไปด้วยหลาย phase ติดต่อกันแล้ว complexity จะเป็น time complexity ที่สูงสุดของ phase หนึ่ง
- เนื่องจาก bottleneck จะอยู่ที่ phase ที่ช้าที่สุด
- ตัวอย่างเช่น โค้ดต่อไปนี้ประกอบด้วย 3 phases ที่มี time complexity  $O(n)$ ,  $O(n^2)$  และ  $O(n)$
- ดังนั้นคอขวดจะเป็น  $O(n^2)$

```

for (int i = 1; i ≤ n; i++) {
 // code O(n)
}

for (int i = 1; i ≤ n; i++) {
 for (int j = 1; j ≤ n; j++) {
 // code O(n^2)
 }
}

for (int i = 1; i ≤ n; i++) {
 // code O(n)
}

```

### 10.13. กรณีหลายตัวแปร

- บางครั้ง time complexity จะขึ้นอยู่กับหลายปัจจัย
- ซึ่งเราสามารถใช้อย่างตัวแปรแทนได้

- ตัวอย่างเช่น time complexity ของโค้ดด้านล่างเป็น  $O(nm)$

```
for (int i = 1; i ≤ n; i++) {
 for (int j = 1; j ≤ m; j++) {
 // code
 }
}
```

## 10.14. Recursion

- time complexity ของฟังก์ชัน recursive จะขึ้นอยู่กับจำนวนครั้งที่ฟังก์ชันนั้นถูกเรียกและ time complexity ของการเรียกฟังก์ชันแต่ละครั้ง
- ซึ่งจะนำสองค่านี้มาคูณกัน
- ยกตัวอย่างเช่น

```
void f(int n) {
 if (n == 1) return;
 f(n-1);
}
```

- การเรียกฟังก์ชัน  $f(n)$  จะเรียกฟังก์ชันทั้งหมด  $n$  ครั้ง และแต่ละครั้งจะใช้ time complexity  $O(1)$
- ดังนั้น time complexity รวมคือ  $O(n) \cdot O(1) = O(n)$
- อีกตัวอย่าง เช่น

```
void g(int n) {
 if (n == 1) return;
 g(n-1);
 g(n-1);
}
```

- ในกรณีนี้ การเรียกฟังก์ชันแต่ละครั้งจะเรียกเพิ่มอีกสองครั้ง ยกเว้นกรณี  $n = 1$
- พิจารณาสิ่งที่เกิดขึ้นจากการเรียก  $g$  ด้วยพารามิเตอร์  $n$
- ตารางด้านล่างแสดงจำนวนการเรียก function ที่เกิดขึ้น

|               |                 |       |        |   |           |   |          |   |  |
|---------------|-----------------|-------|--------|---|-----------|---|----------|---|--|
| function call | number of calls | —   — | $g(n)$ | 1 | $g(n-1)$  | 2 | $g(n-2)$ | 4 |  |
| $c...$        | $c...$          |       | $g(1)$ |   | $2^{n-1}$ |   |          |   |  |

- จากตารางนี้ time complexity คือ  $1 + 2 + 4 + c... + 2^{n-1} = 2^n - 1 = O(2^n)$

## 10.15. Complexity classes

- รายการต่อไปนี้รวม time complexity ของอัลกอริทึมที่เจอบ่อย
- $O(1)$  constant-time algorithm
  - เวลาที่ใช้ในการรันจะเป็น constant ซึ่งไม่สนใจขนาดของ input
  - ยกตัวอย่างเช่นอัลกอริทึมที่มีสูตรคำนวณค่าตอบโดยไม่ต้องทำซ้ำใด ๆ
- $O(\log n)$  logarithmic algorithm
  - อัลกอริทึมแบบ **logarithmic** ส่วนใหญ่จะหารครึ่งขนาดของ input ที่จำเป็นต้องพิจารณาในแต่ละขั้นตอน
  - เวลาที่ใช้ในการรันจะเป็น logarithmic เนื่องจาก  $\log_2 n$  เท่ากับจำนวนครั้งที่หาร  $n$  ด้วย 2 จนกว่าจะเป็น 1
- $O(\sqrt{n})$  square root algorithm



- ช้ากว่า  $O(\log n)$  แต่เร็วกว่า  $O(n)$
- เป็นกรณีที่หารขนาด input  $n$  ด้วย  $\text{sqrt } n = \frac{n}{\text{sqrt } n}$
- $O(n)$  linear algorithm
  - อัลกอริทึมแบบ **linear** จะวนใน input เป็นจำนวนครั้งที่แน่นอน
  - ส่วนใหญ่แล้วจะเป็น time complexity ที่ค่อนข้างดี เนื่องจากปกติแล้วจะจำเป็นต้องอ่านค่าทุกค่าใน input
- $O(n \log n)$ 
  - ส่วนใหญ่จะเกิดจากการเรียงข้อมูล
  - หรือ data structure ที่ใช้  $O(\log n)$  ในแต่ละ operation
- $O(n^2)$  quadratic algorithm
  - ส่วนใหญ่ใช้ลูปสองชั้น
  - เช่นการพิจารณาทุกคู่ของ input
- $O(n^3)$  cubic algorithm
  - ลูปสามชั้น
- $O(2^n)$ 
  - การพิจารณาทุก subsets ของ input
  - ตัวอย่างเช่น subsets ของ  $\{1, 2, 3\}$  คือ  $\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}$  และ  $\{1, 2, 3\}$ .
- $O(n!)$ 
  - การพิจารณาทุกการจัดเรียงของ input
  - ตัวอย่างเช่นการจัดเรียงของ  $\{1, 2, 3\}$  คือ  $(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2)$  and  $(3, 2, 1)$ .

## 10.16. การประเมินประสิทธิภาพ

- จากการคำนวณ time complexity ของอัลกอริทึมนั้นทำให้สามารถตรวจสอบประสิทธิภาพของอัลกอริทึมก่อนการ implement ได้
- จุดอ้างอิงคร่าวๆ ให้คาดว่า C++ สามารถรันได้ 1,000,000,000 (พันล้าน) คำสั่งต่อหนึ่งวินาที
- ตัวอย่างเช่นถ้าข้อจำนวนของโจทย์นั้นให้เวลาในการคำนวณหนึ่งวินาทีและ  $n = 10^5$
- ถ้า time complexity เป็น  $O(n^2)$  อัลกอริทึมนี้อาจจะรันทั้งหมด  $(10^5)^2 = 10^{10}$  คำสั่ง
- ซึ่งโดยคร่าวใช้เวลาประมาณสิบล้านวินาที ซึ่งจะมากเกินไปสำหรับโจทย์ข้อนี้
- ในอีกทางหนึ่ง หากว่าเรารู้ขนาด input แล้ว เราสามารถเดา time complexity ของอัลกอริทึมที่ใช้แก้ปัญหาได้เช่นกัน
- ตารางต่อไปนี้จะบอกการประเมินที่พบบ่อยในกรณีที่ให้เวลาในการคำนวณหนึ่งวินาที
- ยกตัวอย่างเช่น หากขนาดของ input เป็น  $n = 10^5$  time complexity ที่ต้องการจากอัลกอริทึมก็คือ  $O(n)$  หรือ  $O(n \log n)$
- ด้วยความรู้นี้จะทำให้คิดอัลกอริทึมได้ง่ายขึ้นจากการตัดอัลกอริทึมที่ใช้เวลานานออก

| input size | required time complexity | — | — | |  $N! = 10$  |  $O(N!)$  | |  $N! = 20$  |  $O(2^N)$  |  
 | |  $N! = 500$  |  $O(N^3)$  | |  $N! = 5000$  |  $O(N^2)$  | |  $N! = 10^6$  |  $O(N \log\{N\})$  or  $O(N)$  | |  $N$  is large |  $O(1)$  or  $O(\log\{N\})$  |

### 10.17. ตัวแปร constant

- อย่างไรก็ตาม ควรจำว่า time complexity นั้นแค่เป็นการประมาณประสิทธิภาพเท่านั้นเพราะไม่คิด **constant factors**
- ตัวอย่างเช่น อัลกอริทึมที่ใช้เวลา  $O(n)$  อาจมีการคำนวณทั้งหมด  $\frac{n}{2}$  หรือ  $5n$  คำสั่ง
- ซึ่งเป็นสิ่งสำคัญต่อเวลาที่ใช้จริงในการรันอัลกอริทึม

### 10.18. space complexity

- ใช้ Big O Notation เป็นเครื่องมือเช่นกัน
- เปลี่ยนจากการนับขนาดเวลา เป็นนับขนาด memory เช่น
  - ขนาดของ array ที่เราประกาศ

## 11. Priority Queue

### 11.1. คำอธิบาย

- Priority Queue เป็นส่วนเสริมของ queue โดยเพิ่มคุณสมบัติดังนี้
  - ทุก ๆ สมาชิกจะมี priority ของตนเอง
  - สมาชิกที่มี priority สูงสุดจะถูก dequeue ก่อนสมาชิกที่มี priority ต่ำกว่า
  - หากมีสมาชิกที่ priority เท่ากัน จะถูกนำออกตามลำดับเข้าของ queue
- ใน priority queue ด้านล่าง สมาชิกที่มีค่า ASCII มากจะมี priority สูง

 (https://media.geeksforgeeks.org/wp-content/cdn-uploads/Priority-Queue-min-1024x512.png)

<https://media.geeksforgeeks.org/wp-content/cdn-uploads/Priority-Queue-min-1024x512.png>

### 11.2. Operations

- `insert(item, priority)` : เพิ่มสมาชิกพร้อมระบุ priority
- `getHighestPriority()` : คืนค่าสมาชิกที่มี priority สูงสุด
- `deleteHighestPriority()` : นำออกสมาชิกที่มี priority สูงสุด

### 11.3. Implementation

- Heap: สามารถใช้ heap ในการสร้าง priority queue ได้
- STL: ใน STL นั้นมี `priority_queue` ให้ใช้ได้เลย

### 11.4. ตัวอย่าง (STL)

```
#include <bits/stdc++.h>

using namespace std;

int main() {
 priority_queue<int, vector<int>, greater<int>> > q;
 q.push(3);
 q.push(2);
 q.push(15);
 q.push(5);
 q.push(4);
 q.push(45);
 printf("top %d\n", q.top());
 q.pop();
 printf("top %d\n", q.top());
 q.pop();
 printf("top %d\n", q.top());
 return 0;
}
```

## 12. Set & Map

### 12.1. คำอธิบาย

`set` และ `map` ใน STL นั้นมีความคล้ายกันในแง่ที่ทั้งคู่ใช้โครงสร้างข้อมูล Red Black Tree (self balancing BST ประเภทหนึ่ง) โดยใช้เวลาในการ search, insert และ delete เพียง  $O(\log\{n\})$

### 12.2. ข้อแตกต่าง

- `set` ใช้เก็บ keys แต่ `map` ใช้เก็บคู่ key-value
- ตัวอย่างเช่น หากเราต้องการแสดงค่าสมาชิกที่แตกต่างกันเท่านั้น เราสามารถใช้ `set` ได้เนื่องจากต้องการเก็บแค่ key
- แต่ถ้าหากเราต้องการเก็บความถี่ของแต่ละสมาชิกด้วย เราควรใช้ `map` เพื่อเก็บ key ตามสมาชิกโดย value เป็นความถี่

### 12.3. Set example

```
// CPP program to demonstrate working of set
#include <bits/stdc++.h>
using namespace std;

int main() {
 set<int> s1;
 // self-balancing binary search tree
 // binary search tree ที่มีความสูงไม่เกิน 0(log n) เสมอ
 s1.insert(2); // log(n)
 s1.insert(10);
 s1.insert(5);
 s1.insert(3);
 s1.insert(6);

 cout << "Elements in set:\n";
 for (auto it : s1)
 cout << it << " "; // Sorted

 return 0;
}
```

output

```
Elements in set:
2 3 5 6
```

### 12.4. Map example

```
// CPP program to demonstrate working of map
#include <bits/stdc++.h>
using namespace std;

int main() {
 map<int, int> m;
 // คล้ายๆ dictionary ในภาษา python
 // โครงสร้างเดิมนั้น set ก็คือ self-balancing binary search tree

 m[1] = 2; // Insertion by indexing
```

```

// Insertion of pair by make_pair
m.insert(make_pair(8, 5));

cout << "Elements in m:\n";
for (auto it : m)
 cout << "[" << it.first << ", " << it.second << "]\n"; // Sorted

map<string, int> m2;
m2["tt"] = 5; // log(n)
m2["aw"] = 2399;

cout << "Elements in m2:\n";
for (auto it : m2)
 cout << "[" << it.first << ", " << it.second << "]\n"; // Sorted

return 0;
}

```

output

```

Elements in m:
[1, 2]
[8, 5]
Elements in m2:
[aw, 2399]
[tt, 5]

```

## 13. Heap

### 13.1. คำอธิบาย

- Heap เป็นโครงสร้างข้อมูลต้นไม้ (tree) ที่มีเงื่อนไขพิเศษ และเป็น complete binary tree
- โดยทั่วไปแล้ว heap มีสองประเภท
  - ▶ Max-Heap: ใน max heap นั้น key ที่ root จะมีค่าที่มากที่สุด และคุณสมบัตินี้เป็นจริงในทุก ๆ sub-trees
  - ▶ Min-Heap: ใน min heap นั้น key ที่ root จะมีค่าน้อยที่สุด และคุณสมบัตินี้เป็นจริงในทุก ๆ sub-trees

!([https://www.geeksforgeeks.org/wp-content/uploads/MinHeapAndMaxHeap.png°])

(https://www.geeksforgeeks.org/wp-content/uploads/MinHeapAndMaxHeap.png°)

https://www.geeksforgeeks.org/wp-content/uploads/MinHeapAndMaxHeap.png°

### 13.2. คุณสมบัติ

Binary Heap เป็น Binary Tree ที่มีคุณสมบัติเพิ่มเติมดังนี้

- เป็น complete tree (ทุกระดับจะมีโหนดเต็มระดับยกเว้นระดับสุดท้ายซึ่งระดับสุดท้ายจะต้องมีโหนดทางซ้ายให้ได้มากที่สุด)
- คุณสมบัตินี้ทำให้ Binary Heap เหมาะกับการใช้ array ในการเก็บ
- Binary Heap นั้นเป็น Min Heap หรือ Max Heap
  - ▶ ใน Min Binary Heap นั้น ค่าที่ root จะเป็นค่าน้อยที่สุด และคุณสมบัตินี้ต้องเป็นจริงสำหรับทุก node ใน Binary Tree
  - ▶ Max Binary Heap นั้นแค่เปลี่ยนจากค่าน้อยที่สุดเป็นมากที่สุด

### 13.3. Operations (Min Heap)

- `getMin()` : คืนค่า root ของ Min Heap ได้เลย ซึ่งใช้เวลา  $O(1)$
- `extractMin()` : นำค่าน้อยที่สุดใน MinHeap ออก ซึ่งใช้เวลา  $O(\log\{n\})$  เพราะเมื่อนำ root ออกแล้วจำเป็นต้องปรับโครงสร้างให้ตรงตามคุณสมบัติ heap
- `decreaseKey()` : ลดค่าของโหนดที่ระบุ ซึ่งใช้เวลา  $O(\log\{n\})$  เพราะจำเป็นต้องปรับโครงสร้างให้ตรงตามคุณสมบัติ heap
- `insert()` : เพิ่มโหนดใหม่ ซึ่งใช้เวลา  $O(\log\{n\})$  เพราะสามารถเพิ่มโหนดที่ตำแหน่งสุดท้ายแล้วค่อยปรับโครงสร้างให้ตรงคุณสมบัติ heap
- `delete()` : ลบโหนด ซึ่งใช้เวลา  $O(\log\{n\})$  เพราะสามารถ `decreaseKey()` ให้เป็น  $-\infty$  แล้ว `extractMin()`

### 13.4. ตัวอย่างการ implement

- implement subroutine สองฟังก์ชันคือ
  - ▶ `jom` เป็นการปรับโหนดตำแหน่งที่ระบุให้จมลงไปตามคุณสมบัติของ heap
  - ▶ `loj` เป็นการปรับโหนดตำแหน่งที่ระบุให้ลอยขึ้นตามคุณสมบัติของ heap

```
#include <bits/stdc++.h>
int d[100010] = {};
int cnt = 0, heap[100010] = {};
```

```

using namespace std;

void jom(int i) {
 while (i ≤ cnt / 2) {
 int j = i;
 if (i * 2 < cnt)
 if (d[heap[i * 2]] < d[heap[j]])
 j = i * 2;
 if (i * 2 + 1 < cnt)
 if (d[heap[i * 2 + 1]] < d[heap[j]])
 j = i * 2 + 1;
 if (j == i)
 break;
 swap(heap[i], heap[j]);
 i = j;
 }
}

void loy(int i) {
 while (i > 1) {
 if (d[heap[i / 2]] > d[heap[i]]) {
 swap(heap[i / 2], heap[i]);
 i /= 2;
 } else
 break;
 }
}

void insertKey(int k) {
 // First insert the new key at the end
 d[++cnt] = k;
 heap[cnt] = cnt;

 // Fix the min heap property if it is violated
 loy(cnt);
}

void decreaseKey(int i, int new_val) {
 d[heap[i]] = new_val;
 loy(i);
}

// Method to remove minimum element (or root) from min heap
int extractMin() {
 // Store the minimum value, and remove it from heap
 int root = heap[1];
 heap[1] = heap[cnt];
 cnt--;
 jom(1);
 return d[root];
}

// This function deletes key at index i. It first reduced value to minus
// infinite, then calls extractMin()
void deleteKey(int i) {
 decreaseKey(i, INT_MIN);
}

```

```

 extractMin();
}

int getMin() { return d[heap[1]]; }

void printHeap() {
 for (int i = 1; i ≤ cnt; i++) {
 printf("%d ", d[heap[i]]);
 }
 printf("\n");
}

int main() {
 insertKey(3);
 insertKey(2);
 printHeap();
 deleteKey(1);
 printHeap();
 insertKey(15);
 insertKey(5);
 insertKey(4);
 insertKey(45);
 printHeap();
 printf("extractMin %d\n", extractMin());
 printHeap();
 printf("getMin %d\n", getMin());
 decreaseKey(2, 1);
 printHeap();
 printf("getMin %d\n", getMin());
 return 0;
}

```

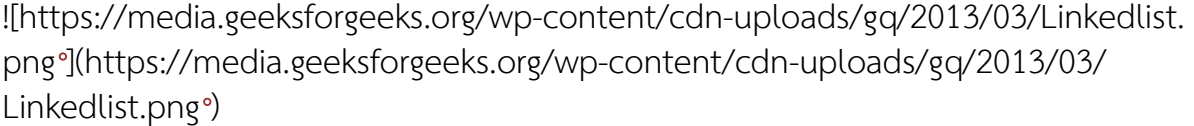


## 14. Review

### 14.1. stack and queue

- stack: first in last out
  - นึกถึงกองจาน
- queue: first in first out
  - คิวทั่วไป

### 14.2. linked list

- เป็น concept ที่เป็นโครงสร้างแนวเดียวกับ array แต่ข้อดีข้อเสียต่างกัน  


### 14.3. dynamic array

- ใช้ vector ของ stl

### 14.4. binary tree



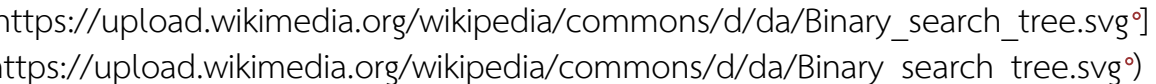
### 14.5. heap



### 14.6. priority queue



### 14.7. binary search tree



### 14.8. set, map

- เหมือน set ทางคณิตศาสตร์
- map
  - เหมือน dictionary ใน Python
  - เหมือน array ที่ index เป็นอะไรก็ได้

### 14.9. graph

- มี cycle
  - tree เป็น graph ที่ไม่มี cycle

## 14.10. hash table

- data structure ที่ใช้บีบข้อมูลให้เล็กลงโดยอาจจะเสียความละเอียดไป
- เช่น บีบ string ให้เหลือแค่ int ผ่าน hash function แต่ก็มีโอกาสชนกันเป็นปัญหา collision

## 15. Stack & Queue

### 15.1. Stack

- เป็นโครงสร้างข้อมูลเชิงเส้น โดยข้อมูลสามารถเข้าและออกได้เพียงทางเดียวที่เรียกว่า **top**
- stack (กองซ้อน) จะใช้หลักเกณฑ์ LIFO (Last In First Out) ก็คือข้อมูลที่ถูกเพิ่มทีหลังสุดจะเป็นข้อมูลที่จะถูกนำออกก่อน
- การเพิ่มข้อมูลลงใน stack จะเรียกว่าการ **push** ส่วนการลบข้อมูลจะเรียกว่าการ **pop**
- ใน stack เราจะมีตัวแปรเก็บตำแหน่งข้อมูลที่เข้าหลังสุดอยู่เสมอ โดยจะเรียกว่า **top**

![https://media.geeksforgeeks.org/wp-content/uploads/geek-stack-1.png°](https://media.geeksforgeeks.org/wp-content/uploads/geek-stack-1.png°)

(https://media.geeksforgeeks.org/wp-content/uploads/geek-stack-1.png°)

### 15.2. ตัวอย่างการใช้ vector เพื่อ implement stack

```
#include <bits/stdc++.h>

using namespace std;

void printvec(vector<int> &a) {
 for (auto x : a)
 printf("%d ", x);
 printf("\n");
}

int main() {
 // stack
 vector<int> a;
 a.push_back(1);
 a.push_back(2);
 a.push_back(3);
 a.push_back(4);
 printvec(a); // 1 2 3 4
 a.pop_back();
 a.pop_back();
 printvec(a); // 1 2
}
```

### 15.3. Queue

- queue (แถวคอย) เป็นโครงสร้างข้อมูลเชิงเส้น โดยข้อมูลจะเข้าได้ทางหนึ่งที่เรียกว่า **rear** และจะถูกนำออกได้ในอีกทางที่เรียกว่า **front**
- queue จะหลักกร (First In First Out) ก็คือข้อมูลที่ถูกนำเข้าแรกสุดจะเป็นข้อมูลที่ถูกนำออกแรกสุด
- การเพิ่มข้อมูลเข้า queue จะเรียกว่า **enqueue** และการลบข้อมูลจะเรียกว่า **dequeue**
- ใน queue นั้นเราจะมีตัวแปรเก็บตำแหน่งสองตำแหน่ง
  - **front** ในการเก็บข้อมูลปัจจุบันที่ถูกนำเข้า **แรก** สุด
  - **rear** ในการเก็บข้อมูลปัจจุบันที่ถูกนำเข้า **หลัง** สุด

![https://media.geeksforgeeks.org/wp-content/uploads/geek-queue-1.png°](https://media.geeksforgeeks.org/wp-content/uploads/geek-queue-1.png°)

(<https://media.geeksforgeeks.org/wp-content/uploads/geek-queue-1.png>)

## 15.4. Stack vs Queue

| Stack | Queue | | — | — | | LIFO | FIFO | | one pointer (top) | two pointers (front, rear) | | push | enqueue | | pop | dequeue | | recursion | sequential |

## 15.5. Deque (double-ended queue)

- ใน STL จะมีโครงสร้างข้อมูลชื่อ deque โดยจะเป็นโครงสร้างข้อมูลเชิงเส้นที่สามารถเพิ่มลบข้อมูลได้ทั้งสองทิศทางด้วยคำสั่ง
  - push\_front
  - push\_back
  - pop\_front
  - pop\_back

## 15.6. ตัวอย่าง deque

```
#include <bits/stdc++.h>

using namespace std;

void printq(deque<int> &a) {
 for (auto x : a)
 printf("%d ", x);
 printf("\n");
}

int main() {
 // double-ended queue
 deque<int> b;
 b.push_back(1);
 b.push_back(2);
 b.push_back(3);
 b.push_back(4);
 printq(b); // 1 2 3 4
 b.pop_front();
 b.pop_front();
 printq(b); // 3 4
 b.pop_back();
 printq(b); // 3
}
```

## 16. Hash Table

### 16.1. Idea

- hash table เป็น data structure ที่เอาไว้สร้าง dictionary abstract data structure
- hash function

(figure from [[https://en.wikipedia.org/wiki/Hash\\_table](https://en.wikipedia.org/wiki/Hash_table)])([https://en.wikipedia.org/wiki/Hash\\_table](https://en.wikipedia.org/wiki/Hash_table))

- หัวใจหลักของการทำ hashing คือการกระจาย entries (key/value pairs) ไปไว้ในช่องต่างๆ ใน array ที่เรากำหนด
- เมื่อเรามี key แล้วกระบวนการนี้ควรจะคิด index ที่ key นี้ควรจะอาศัยอยู่

```
index = f(key, array_size)
```

- ตามปกติแล้วกระบวนการนี้จะมีสองขั้นตอน คือ

```
hash = hashfunc(key) // คิดค่า hash จาก hash function ของ key
index = hash % array_size // mod ค่า hash ด้วย array_size เพื่อป้องกัน index error
```

- ในกรณีนี้ การ hash จะไม่สนใจขนาดของ array แล้วค่อยจำกัดขอบเขตภายหลังด้วยการ modulo (%) ให้ index อยู่ในช่วง  $[0, \text{array\_size} - 1]$

### 16.2. Polynomial hashing

- ตัวอย่าง hash function หนึ่งคือ polynomial hashing ซึ่งมีลักษณะดังนี้

$$(s[0]A^{n-1} + s[1]A^{n-2} + \dots + s[n-1]A^0) \bmod B$$

- โดย  $s[0], s[1], \dots, s[n-1]$  คือค่าของแต่ละตัวอักษรของ  $s$
- และ  $A, B$  เป็นค่าคงที่ที่เรากำหนด

#### 16.2.1. ตัวอย่าง

- คำว่า ALLEY จะมีค่าดังตาราง

|   |   |   |   |   |  |  |  |  |  |  |    |    |    |    |    |
|---|---|---|---|---|--|--|--|--|--|--|----|----|----|----|----|
| A | L | L | E | Y |  |  |  |  |  |  | 65 | 76 | 76 | 69 | 89 |
|---|---|---|---|---|--|--|--|--|--|--|----|----|----|----|----|

- หากกำหนดให้  $A = 3$  และ  $B = 97$  จะทำให้ hash value มีค่าคือ

$$(65 \times 3^4 + 76 \times 3^3 + 76 \times 3^2 + 69 \times 3^1 + 89 \times 3^0) \% 97 = 52$$

- เราก็สามารถกำหนดได้ว่าคำว่า ALLEY ควรจะอยู่ช่องที่ 52

### 16.3. ตัวอย่างโปรแกรม

```
#include <bits/stdc++.h>

using namespace std;

const int A = 3;
const int B = 97;

string hash_table[B];

int hash_function(string s) {
 int sm = 0;
```

```

for (auto x : s) {
 sm *= A;
 sm += int(x);
 sm %= B;
}
return sm;
}

int main() {
 string s = "ALLEY";
 printf("Hash value of %s is %d\n", s.c_str(), hash_function(s));
 hash_table[hash_function(s)] = s;
}

```

ได้ output แบบนี้

Hash value of ALLEY is 52

💡 ข้อสังเกต

</aside>

- hash function ควรจะเป็น deterministic function ซึ่งหมายความว่าถึงเราจะรันกี่ครั้งก็ควรจะได้ค่าเดิมเสมอ
- ไม่อย่างนั้นเราจะไม่สามารถรู้ได้ว่าค่าๆหนึ่งควรจะอยู่ที่ช่องอะไร หากรันสองครั้งแล้วค่าไม่เหมือนกัน
- ตัวอย่างของ nondeterministic function คือการ random ค่าขึ้นมา ทำให้ไม่มีการการันตีว่ารันสองครั้งแล้วจะได้ค่าเดิม

## 16.4. Collision

- เนื่องจากเวลาทำ hashing จะไม่สามารถการันตีได้ว่าค่า hash ของสองค่าที่แตกต่างกันจะแตกต่างกันเสมอ
- เช่นหาก hash value ของ “John Smith” กับ “Sandra Dee” เป็นค่าเดียวกันที่ค่า 152 จะเกิดปัญหาว่าไม่สามารถเก็บทั้งสองค่าในช่องเดียวกันได้
- มีวิธีการแก้ปัญหาหลายแบบ ข้อดีข้อเสียแตกต่างกัน

### 16.4.1. Separate Chaining

![[https://upload.wikimedia.org/wikipedia/commons/thumb/d/d0/Hash\_table\_5\_0\_1\_1\_1\_1\_1\_LL.svg/900px-Hash\_table\_5\_0\_1\_1\_1\_1\_1\_LL.svg.png]](https://upload.wikimedia.org/wikipedia/commons/thumb/d/d0/Hash\_table\_5\_0\_1\_1\_1\_1\_1\_LL.svg/900px-Hash\_table\_5\_0\_1\_1\_1\_1\_1\_LL.svg.png)

(figure from [https://en.wikipedia.org/wiki/Hash\_table](https://en.wikipedia.org/wiki/Hash\_table))

- ทำให้แต่ละช่องเป็น linked list

### 16.4.2. Open addressing

![[https://upload.wikimedia.org/wikipedia/commons/thumb/b/bf/Hash\_table\_5\_0\_1\_1\_1\_1\_0\_SP.svg/760px-Hash\_table\_5\_0\_1\_1\_1\_1\_0\_SP.svg.png]](https://upload.wikimedia.org/wikipedia/commons/thumb/b/bf/Hash\_table\_5\_0\_1\_1\_1\_1\_0\_SP.svg/760px-Hash\_table\_5\_0\_1\_1\_1\_1\_0\_SP.svg.png)

wikimedia.org/wikipedia/commons/thumb/b/bf/Hash\_table\_5\_0\_1\_1\_1\_1\_0\_SP.svg/760px-Hash\_table\_5\_0\_1\_1\_1\_1\_0\_SP.svg.png<sup>o</sup>)

(figure from [[https://en.wikipedia.org/wiki/Hash\\_table](https://en.wikipedia.org/wiki/Hash_table)]<sup>o</sup>)([https://en.wikipedia.org/wiki/Hash\\_table](https://en.wikipedia.org/wiki/Hash_table))

- ขยับไปช่องถัดไป

### 16.4.3. เปรียบเทียบ

- พิจารณาถึงตอนที่เรากำลังตรวจสอบว่ามีข้อมูลดังกล่าวอยู่ในตารางหรือไม่
- พิจารณากรณีที่มีการลบข้อมูลเกิดขึ้น

## 17. Greedy

### 17.1. หลักการ

- วิธีคิดสำหรับปัญหา optimization โดยทั่วไปแล้วจะมีขั้นตอน ซึ่งแต่ละขั้นจะมีตัวเลือกที่แตกต่างกันไป
- สำหรับหลายๆ ปัญหา optimization อาจจะไม่จำเป็นต้องใช้ dynamic programming เพื่อตัดสินใจตัวเลือกที่ดีที่สุด
- วิธีคิดแบบ greedy จะเลือกตัวเลือกที่ดีที่สุด ณ ขณะนั้นๆ เสมอ โดยหวังว่าการเลือกตัวเลือกที่ดีที่สุด ณ local จะนำไปสู่วิธีการแก้ปัญหาที่ดีที่สุด ณ global
- วิธีคิดแบบ greedy จะไม่นำไปสู่วิธีการแก้ปัญหาที่ดีที่สุดเสมอไป มีเพียงบางปัญหาเท่านั้นที่สามารถนำไปใช้ได้
- การพิสูจน์ว่าสามารถใช้วิธีคิดแบบ greedy นั้นทำได้ยาก บางครั้งการลองเขียนโค้ดไปเลยจะสะดวกกว่าการพิสูจน์

### 17.2. ปัญหาการทอนเหรียญ

พิจารณาปัญหาที่ต้องการทอนเงินมูลค่า  $n$  โดยใช้เหรียญที่มีมูลค่า  $\{c_1, c_2, c_3, \dots, c_k\}$  จำนวนเหรียญที่น้อยที่สุดที่ต้องใช้เป็นเท่าไร?

#### 17.2.1. ตัวอย่าง

มูลค่าของเหรียญคือ

$\{1, 2, 5, 10, 20, 50, 100, 200\}$

และ  $n = 520$

#### 17.2.2. Greedy approach

ตัวอย่างวิธีคิดแบบ greedy คือเลือกเหรียญที่มีมูลค่ามากที่สุดก่อนเสมอ จนกว่าเราจะสามารถรวมมูลค่าได้ถูกต้อง

**คำเตือน** วิธีคิดแบบ greedy ไม่จำเป็นจะเป็นวิธีที่ดีที่สุดเสมอไป กรณีนี้ขึ้นอยู่กับประเภทของเหรียญว่าสามารถใช้วิธี greedy ได้หรือไม่

### 17.3. ปัญหาการจัดตารางเวลา

พิจารณาปัญหาที่ต้องการเรียง  $n$  งานซึ่งมีเวลาเริ่มและเวลาจบแตกต่างกันไป โดยต้องการให้สามารถจัดเรียงงานให้ได้จำนวนมากที่สุด

#### 17.3.1. ตัวอย่าง

| งาน | เวลาเริ่ม | เวลาจบ | | — | — | — | | A | 1 | 3 | | B | 2 | 5 | | C | 3 | 9 | | D | 6 | 8 |

#### 17.3.2. คำอธิบาย

แนวคิดที่เลือกงานถัดไปที่เลิกเร็วที่สุดเสมอ สามารถอ้างได้ว่าหากเราเลือกงานที่เลิกช้ากว่างานที่เราเลือก เราจะได้จำนวนงานอย่างมากที่สุดเท่ากับจำนวนงานที่เราเลือกด้วยวิธีเดิม เพราะฉะนั้นการเลือกงานที่เลิกช้ากว่าจะไม่สามารถให้คำตอบที่ดีกว่าได้



## 17.4. เพิ่มเติม

- การบรรยายวิชา การออกแบบและวิเคราะห์อัลกอริทึม

โดย สมชาย ประสิทธิ์จูตระกูล

- [อัลกอริทึม : 5.1 อัลกอริทึมแบบละโมบ - แนวคิด](<https://youtu.be/TDZ37p9CH5Y>)
- [อัลกอริทึม : 5.2 อัลกอริทึมแบบละโมบ - การจัดกิจกรรม]([https://youtu.be/t\\_OX-QqZxzE](https://youtu.be/t_OX-QqZxzE))
- [อัลกอริทึม : 5.3 อัลกอริทึมแบบละโมบ - Knapsack](<https://youtu.be/jSAmgSPFsXY>)
- [อัลกอริทึม : 5.4 อัลกอริทึมแบบละโมบ - รหัสฮัฟฟ์แมน](<https://youtu.be/OD2ZnPSXFLA>)

## 18. Array Manipulation

### 18.1. คำอธิบาย

[Dynamic Range Sum Queries](https://cses.fi/problemset/task/1648/°)

- Fenwick Tree
  - [https://csacademy.com/lesson/fenwick\_trees/°](https://csacademy.com/lesson/fenwick\_trees/°)
  - [https://visualgo.net/en/fenwicktree?slide=1°](https://visualgo.net/en/fenwicktree?slide=1°)
- Segment Tree
  - [https://csacademy.com/lesson/segment\_trees/°](https://csacademy.com/lesson/segment\_trees/°)
  - [https://visualgo.net/en/segmenttree?slide=1°](https://visualgo.net/en/segmenttree?slide=1°)
- Query time vs Preprocess time
- Hash [https://visualgo.net/en/hashtable°](https://visualgo.net/en/hashtable°)
- Heap
  - [https://csacademy.com/lesson/heaps/°](https://csacademy.com/lesson/heaps/°)
  - [https://csacademy.com/lesson/heap\_application\_merge\_k\_sorted\_lists/°](https://csacademy.com/lesson/heap\_application\_merge\_k\_sorted\_lists/°)
  - [https://visualgo.net/en/heap?slide=1°](https://visualgo.net/en/heap?slide=1°)
- Union-find Disjoint Sets [https://visualgo.net/en/ufds?slide=1°](https://visualgo.net/en/ufds?slide=1°)

## 19. Search

### 19.1. หลักการ

- เป็นวิธีทั่วไปที่แก้ปัญหาก็ได้แทบจะทุกปัญหา
- เป็นการ brute force หาทุกคำตอบที่เป็นไปได้

### 19.2. Generating Permutations

เขียน recursive function เพื่อ generate ทุก permutation ที่เป็นไปได้

### 19.3. Graph Traversal

[visualgo](<https://visualgo.net/en/dfsbfbs?slide=1>)

- BFS
- DFS
  - preorder
  - inorder
  - postorder
- Topological Sort

### 19.4. Binary Search Tree

[visualgo](<https://visualgo.net/en/bst?slide=1>)

- create
- search(v)
- insert(v)
- remove(v)

### 19.5. Meet in the middle

- เป็นเทคนิคที่ถ้า search จากต้นทางแล้ว space ใหญ่เกินไป ให้ search จากปลายทางแล้วมาเจอกันตรงกลาง

## 20. Dynamic Programming

### 20.1. หลักการ

- Dynamic programming (DP) เป็นเทคนิคที่รวมการค้นหาลำหัดเข้ากั้บวิธีคิดแบบ greedy
- DP จะแก้ปัญหาลำหัดหากปัญหาลำหัดสามารถแบ่งออกมามีปัญหาลำหัดย่อยที่ทับซ้อนกันและสามารถแก้ได้อย่างมีประสิทธิภาพ
- ประโยชน์หลักของ DP คือ
  - หาวิธีแก้ปัญหาลำหัด optimal
  - หาจำนวนวิธีแก้ปัญหาลำหัด
- โ้เดียว DP เป็นโ้เดียวที่ง่ายแต่ส่วนที่ยากจะเป็นการนำโ้ไปใช้กับปัญหาลำหัดจริง

ซึ่งต้องใช้ประสิทธิภาพ

- Backtracking สร้างตารางเก็บที่มามีคำตอบหากโจทย์ต้องการ

### 20.2. ปัญหาการทอนเหรียญ (ปัญหาลำหัดเดียวกับ Greedy)

พิจารณาปัญหาลำหัดที่ต้องการทอนเงินมูลค่า  $n$  โดยใช้เหรียญที่มีมูลค่า  $\{c_1, c_2, \dots, c_k\}$  จำนวนเหรียญที่น้อยที่สุดที่ต้องใช้เป็นเท่าไร?

หากมูลค่าของเหรียญเป็น

$\{1, 3, 4\}$

| มูลค่ารวม  $n$  | จำนวนเหรียญที่น้อยที่สุด || — | — || 0 | 0 || 1 | 1 || 2 | 2 || 3 | 1 || 4 | 1 || 5 | 2 || 6 | 2 || 7 | 2 || 8 | 2 || 9 | 3 || 10 | 3 |

นิยาม  $d_n$  ให้เก็บค่าแทนจำนวนเหรียญที่น้อยที่สุดที่ใช้ทอนเงินมูลค่า  $n$

จะได้ว่า

$$d_i = m \in \{(d_{i-1} + 1, d_{i-3} + 1, d_{i-4} + 1)\}$$

ซึ่งเป็น recursive case โดย base case คือ  $d_0 = 0$

### 20.3. Longest Increasing Subsequence

[[https://cp-algorithms.com/sequences/longest\\_increasing\\_subsequence.html](https://cp-algorithms.com/sequences/longest_increasing_subsequence.html)]

([https://cp-algorithms.com/sequences/longest\\_increasing\\_subsequence.html](https://cp-algorithms.com/sequences/longest_increasing_subsequence.html))

### 20.4. Review

#### 20.4.1. คืออะไร?

- คำนวณค่าใน state หนึ่งจาก state ก่อนหน้า โดยไม่ต้องไปคำนวณใหม่
- recursion ที่เรียก overlapping sub-problem

แล้วเอามาใช้ได้เลยโดยไม่ต้องไปคำนวณใหม่

- แก้โจทย์โดยการจำ เล่นกับหน่วยความจำ เพื่อที่จะได้ไม่ต้องคำนวณซ้ำ
- หาค่าปัจจุบันจากค่าก่อนหน้า แล้วเราหาเคสเล็ก

#### 20.4.2. มีอะไรบ้าง?

- quicksum  $dp[i] = dp[i-1] + a[i]$

- knapsack  $dp[i] = \max(dp[i-w_i] + c_i)$
- fibonacci  $dp[i] = dp[i-2] + dp[i-1]$
- dijkstra  $dp[v] = \min(dp[u] + w)$
- LIS longest increasing subsequence
- pascal (nCr)  $dp[i][j] = dp[i-1][j-1] + dp[i-1][j]$
- coin change  $dp[i] = f(dp[i-x_j])$
- matrix chain  $N^3$ ,  $dp[i][j] = \min(f(dp[i][k], dp[k+1][j]))$
- LCS longest common subsequence
- LCA lowest common ancestor
- #paths pascal
- longest common substring lis
- matrix exponential divide&conquer/bitwise
  - $x^9$
  - $9 = \{1001\}_2$
  - $x^1 * x^8$
- KMP Knutt-Morris-Pratt
- z-function
- floyd-warshall matrix chain  $N^3$ ,  $dp[i][j] = \min(f(dp[i][k], dp[k][j]))$
- Manacher algorithm
- TSP travelling salesman  $2^n * n$ ,  $dp[i][j]$  โดยที่  $i$  เป็น set ของเมืองที่เคยผ่านไปแล้ว และ  $j$  เป็นเมืองสุดท้ายที่อยู่ตอนนั้น
- tribonacci

### 20.4.3. example

#### 20.4.3.1. fibonacci

$$f(n) = f(n-1) + f(n-2)$$

- 1 1 2 3 5 8 11 ..
- $f(10) = f(9) + f(8)$
- $f(9) = f(8) + f(7)$

for (i: 2 → n)  $dp[i] = dp[i-1] + dp[i-2]$

#### 20.4.3.2. LIS

หาลำดับย่อยที่ยาวที่สุดที่เป็น increasing subsequence

1 5 3 2 8 9 3 6 → 1 2 3 6 / 1 2 8 9 / 1 5 8 9 / = 4

$N^2$  solution:

define

state:  $dp[i]$  = ความยาวสูงสุดของ increasing subsequence เมื่อพิจารณาช่องที่ 0-i แล้วลงท้ายด้วยเลข  $a[i]$

คำตอบสุดท้าย  $\max(dp[1:N])$

update state: new input:  $x$ , พิจารณา  $a[j] < x$ , แล้วเก็บค่า  $dp[i] = \max(dp[j] + 1)$

backtrack: + ให้เก็บด้วย j เป็นเท่าไร

0 1 2 3 4 5 6 7 8

a : 1 5 3 2 8 9 3 6  
dp : 0 1 2 2 2 3 4 3 4  
bt : 0 0 1 1 1 4 5 4 7

$N \log\{N\}$  solution:

define state:  $dp[i]$  = ค่าที่น้อยที่สุดที่จบที่ค่าสุดท้ายของ increasing subsequence ความยาว  $i$

update state: new input:  $x$  ใส่ตั้งแต่  $dp[j: 0 \rightarrow mx]$

- ถ้าค่า  $dp[j]$  มากกว่า  $x$  และ  $j$  มีค่าน้อยที่สุด,  $dp[j] = x$ ,
- แต่ถ้าหาไม่ได้เลย ก็ให้  $dp[mx+1] = x$

คำตอบสุดท้าย  $mx$

$dp[0] = 0$   
 $dp[1] = 1$   
 $dp[2] = 2$   
 $dp[3] = 3$   
 $dp[4] = 6$   
 $dp[5] =$

## 20.5. เพิ่มเติม

- [An Introduction to Dynamic Programming](http://tcpc.me/2019/01/28/an-introduction-to-dynamic-programming.html<sup>๑</sup>) (aquablitz11)
- [Dynamic Programming by Aj. Nattee](https://github.com/nattee/ioi-training-note/blob/main/Dynamic%20Programming/01-dynamic%20programming.pdf<sup>๑</sup>)
- การบรรยายวิชา การออกแบบและวิเคราะห์อัลกอริทึม

โดย สมชาย ประสิทธิ์จิตรระกุล

- [อัลกอริทึม : 4.1 กำหนดการพลวัต - แนวคิด](https://youtu.be/kUwIJBm7uKM<sup>๑</sup>)
- [อัลกอริทึม : 4.2 กำหนดการพลวัต -  $C(n,k)$ ](https://youtu.be/a7fv36U-Kss<sup>๑</sup>)
- [อัลกอริทึม : 4.3 กำหนดการพลวัต - ลำดับย่อยร่วมยาวสุด : บนลงล่าง](https://youtu.be/ONcpgeQ5bHY<sup>๑</sup>)
- [อัลกอริทึม : 4.4 กำหนดการพลวัต - ลำดับย่อยร่วมยาวสุด : ล่างขึ้นบน](https://youtu.be/ZOd8JXbBDKc<sup>๑</sup>)
- [อัลกอริทึม : 4.5 กำหนดการพลวัต - ลักษณะของปัญหา](https://youtu.be/SQhZrqwC-co<sup>๑</sup>)
- [อัลกอริทึม : 4.6 กำหนดการพลวัต - ถุงเป้แบบ 0/1 : บนลงล่าง](https://youtu.be/ib-sAXfvzZo<sup>๑</sup>)
- [อัลกอริทึม : 4.7 กำหนดการพลวัต - ถุงเป้แบบ 0/1 : ล่างขึ้นบน](https://youtu.be/VILNeEZkTKk<sup>๑</sup>)
- [อัลกอริทึม : 4.8 กำหนดการพลวัต - ผลรวมติดกันสูงสุด](https://youtu.be/y2JqDPYdT3A<sup>๑</sup>)
- [อัลกอริทึม : 4.9 กำหนดการพลวัต - แบ่งข้อความไทย : บนลงล่าง](https://youtu.be/Ov\_OUU6vHxA<sup>๑</sup>)
- [อัลกอริทึม : 4.10 กำหนดการพลวัต - แบ่งข้อความไทย : ล่างขึ้นบน](https://youtu.be/bhdX3WReaEU<sup>๑</sup>)

- [อัลกอริทึม : 4.11 กำหนดการพลวัต - การคูณเมทริกซ์ : บนลงล่าง](<https://youtu.be/ZdUyLGsE5gM>)
- [อัลกอริทึม : 4.12 กำหนดการพลวัต - การคูณเมทริกซ์ : ล่างขึ้นบน](<https://youtu.be/iFepj73HyFk>)

## 21. Ad-hoc

### 21.1. หลักการ

- โจทย์ประเภทนี้เป็นโจทย์ที่เน้นทักษะการ implementation

### 21.2. String

- [Rabin-Karp](<https://cp-algorithms.com/string/rabin-karp.html>)
- [Knuth-Morris-Pratt](<https://cp-algorithms.com/string/prefix-function.html>)



## 22. Divide & Conquer

### 22.1. หลักการ

- ประกอบด้วยขั้นตอนหลักสามขั้นตอน
  - Divide แบ่งปัญหาให้เป็นปัญหาเดียวกันที่ขนาดเล็กลง
  - Conquer แก้ปัญหาย่อยๆ ให้ได้
  - Combine

รวบรวมคำตอบของปัญหาย่อยที่ถูกแก้แล้วมารวมกันเพื่อแก้ปัญหาเดิมที่ใหญ่กว่า

- ปัญหาที่ยังสามารถย่อยต่อเพื่อแก้ปัญหาได้เรียกว่า recursive case
- ปัญหาที่ขนาดเล็กเพียงพอที่จะแก้ปัญหาได้โดยไม่ต้องย่อยต่อเรียกว่า

base case

- ปัญหาส่วนใหญ่จะใช้เวลา  $O(N^2 \log\{N\})$  โดย  $\log\{N\}$

มาจากการแบ่งปัญหาเป็นครึ่งๆ ไปเรื่อยๆ

#### 22.1.1. General Pseudocode

```
DAC(p):
 if is_base_case(p):
 solve(p)
 else:
 divide p into p_1, p_2, ..., p_k
 apply DAC(p_1), DAC(p_2), ..., DAC(p_k)
 combine(DAC(p_1), DAC(p_2), ..., DAC(p_k))
```

### 22.2. Binary Search

การหาจำนวน (หรือคำตอบที่ต้องการ) ในอาร์เรย์ที่มีการเรียงแล้ว โดยตัวอัลกอริธึมจะตรวจสอบกับตัวเลขตรงกลาง ก่อนที่จะแบ่งอาร์เรย์ออกเป็นสองฝั่งเพื่อหาตัวเลขนั้นอีกครั้ง หากตัวเลขที่กำลังหาอยู่มากกว่าตัวเลขตรงกลาง ก็จะไปหาจำนวนนั้นในอาร์เรย์ฝั่งขวา แต่ถ้าหากตัวเลขที่กำลังหาอยู่น้อยกว่าตัวเลขตรงกลาง ก็จะไปหาจำนวนนั้นในอาร์เรย์ฝั่งซ้าย

#### 22.2.1. Pattern

```
int lo = 0, hi = N;
while (lo ≤ hi) {
 int mi = (lo + hi) / 2; // ระวัง overflow ตอน lo+hi
 if (A[mi] > target) {
 hi = mi - 1;
 } else {
 lo = mi + 1;
 }
}
```

#### 22.2.2. STL

- upper\_bound
- lower\_bound

#### 22.2.3. Binary Search คำตอบ

- ปัญหาบางอย่างสามารถสมมุติคำตอบแล้ว check ได้เร็ว

- หากคำตอบมี pattern เช่น ถ้าค่าน้อยสามารถทำได้แล้วค่าที่มากกว่าจะทำได้ด้วยเสมอ ก็จะสามารถ binary search ได้

## 22.3. More on Binary Search

- [สอน Binary Search ฉบับสมบูรณ์โคตรๆ](https://aquablitz11.github.io/2019/04/12/complete-bsearch-tutorial.html<sup>๑</sup>)
- [https://csacademy.com/lesson/binary\_search/<sup>๑</sup>](https://csacademy.com/lesson/binary\_search/<sup>๑</sup>)

## 22.4. Closest Pair

[https://www.geeksforgeeks.org/closest-pair-of-points-using-divide-and-conquer-algorithm/<sup>๑</sup>](https://www.geeksforgeeks.org/closest-pair-of-points-using-divide-and-conquer-algorithm/<sup>๑</sup>)

## 22.5. ปัญหาเลขยกกำลัง

พิจารณาวิธีหาค่าของ  $A^N$  หากทำการคูณเลขตรงๆ จะเป็น  $O(N)$

ถ้าเราแบ่ง

- Recursive case
  - $A^N = A^{\lfloor \frac{N}{2} \rfloor} \cdot A^{\lfloor \frac{N}{2} \rfloor}$  เมื่อ  $N \bmod 2 = 0$
  - $A^N = A^{\lfloor \frac{N}{2} \rfloor} \cdot A^{\lfloor \frac{N}{2} \rfloor + 1}$  เมื่อ  $N \bmod 2 = 1$
- Base case
  - $A^N = A$  เมื่อ  $N = 1$
  - $A^N = 1$  เมื่อ  $N = 0$

## 22.6. จำนวน Fibonacci

จำนวนฟีโบนัชชี (เขียนแทนด้วย  $F_n$ ) คือลำดับจำนวนที่ถูกระบุนิยามจากผลบวกของจำนวนฟีโบนัชชีสองตัวก่อนหน้า โดยกำหนดให้จำนวนฟีโบนัชชีสองตัวแรกเป็น 0 และ 1 ตามลำดับ นั่นคือ

โดยจะสามารถเขียนลำดับได้เป็น

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...

เพราะฉะนั้นแล้ว Base Case ก็คือ  $0l = 1l = 1$  และ Recursive Case ก็คือ  $ig = 2$

## 22.7. Merge Sort

```
void merge(int array[], int const left, int const mid, int const right) {
 auto const subArrayOne = mid - left + 1;
 auto const subArrayTwo = right - mid;

 // Create temp arrays
 auto *leftArray = new int[subArrayOne], *rightArray = new int[subArrayTwo];

 // Copy data to temp arrays leftArray[] and rightArray[]
 for (auto i = 0; i < subArrayOne; i++)
 leftArray[i] = array[left + i];
 for (auto j = 0; j < subArrayTwo; j++)
 rightArray[j] = array[mid + 1 + j];
```

```

auto indexOfSubArrayOne = 0, // Initial index of first sub-array
 indexOfSubArrayTwo = 0; // Initial index of second sub-array
int indexOfMergedArray = left; // Initial index of merged array

// Merge the temp arrays back into array[left..right]
while (indexOfSubArrayOne < subArrayOne && indexOfSubArrayTwo < subArrayTwo) {
 if (leftArray[indexOfSubArrayOne] ≤ rightArray[indexOfSubArrayTwo]) {
 array[indexOfMergedArray] = leftArray[indexOfSubArrayOne];
 indexOfSubArrayOne++;
 } else {
 array[indexOfMergedArray] = rightArray[indexOfSubArrayTwo];
 indexOfSubArrayTwo++;
 }
 indexOfMergedArray++;
}
// Copy the remaining elements of
// left[], if there are any
while (indexOfSubArrayOne < subArrayOne) {
 array[indexOfMergedArray] = leftArray[indexOfSubArrayOne];
 indexOfSubArrayOne++;
 indexOfMergedArray++;
}
// Copy the remaining elements of
// right[], if there are any
while (indexOfSubArrayTwo < subArrayTwo) {
 array[indexOfMergedArray] = rightArray[indexOfSubArrayTwo];
 indexOfSubArrayTwo++;
 indexOfMergedArray++;
}
}

```

## 22.8. Square Root Decomposition

- [<https://usaco.guide/plat/sqrt?lang=cpp>](<https://usaco.guide/plat/sqrt?lang=cpp>)
- [[https://cp-algorithms.com/data\\_structures/sqrt\\_decomposition.html](https://cp-algorithms.com/data_structures/sqrt_decomposition.html)]([https://cp-algorithms.com/data\\_structures/sqrt\\_decomposition.html](https://cp-algorithms.com/data_structures/sqrt_decomposition.html))

## 22.9. เพิ่มเติม

- การบรรยายวิชา การออกแบบและวิเคราะห์อัลกอริทึม

โดย สมชาย ประสิทธิ์จูตระกูล

- [อัลกอริทึม : 3.1 การแบ่งแยกและเอาชนะ - แนวคิด]([https://youtu.be/X8z0E-I\\_JM](https://youtu.be/X8z0E-I_JM))
- [อัลกอริทึม : 3.2 การแบ่งแยกและเอาชนะ - การยกกำลัง](<https://youtu.be/9l7A8yZSP0Q>)
- [อัลกอริทึม : 3.3 การแบ่งแยกและเอาชนะ - การคูณจำนวนเต็ม](<https://youtu.be/rsp0dWLPQhw>)
- [อัลกอริทึม : 3.4 การแบ่งแยกและเอาชนะ - การคูณเมทริกซ์](<https://youtu.be/KdcZWXseSvY>)
- [อัลกอริทึม : 3.5 การแบ่งแยกและเอาชนะ - การเรียงลำดับแบบผสาน](<https://youtu.be/J4GDbT2h3PE>)

- [อัลกอริทึม : 3.6 การแบ่งแยกและเอาชนะ - การเรียงลำดับแบบเร็ว](<https://youtu.be/fp8k4hWNjE4>)
- [อัลกอริทึม : 3.7 การแบ่งแยกและเอาชนะ - ตัวน้อยสุดอันดับ k](<https://youtu.be/07kz0R29NTk>)
- [อัลกอริทึม : 3.8 การแบ่งแยกและเอาชนะ - คู่จุดที่ใกล้กันสุด](<https://youtu.be/s8BOByAlxAk>)
- [อัลกอริทึม : 3.9 การแบ่งแยกและเอาชนะ - ค่ามากที่สุด-ค่าน้อยสุด](<https://youtu.be/g-tPuncBNAU>)
- [อัลกอริทึม : 3.10 การแบ่งแยกและเอาชนะ - การหาดาวเด่น](<https://youtu.be/TWkJbnvbFVw>)

## 23. Graph Algorithm

### 23.1. Day 1

#### 23.1.1. Search in Graph

- Depth-First Search
  - <https://grader.mwit.ac.th/problem/vance><sup>◦</sup>  
hint</summary>dfs from all node</details>
- Breadth-First Search
  - [https://grader.mwit.ac.th/problem/toi17\\_wall](https://grader.mwit.ac.th/problem/toi17_wall)<sup>◦</sup>  
hint</summary>dfs/bfs โจทย์ซับซ้อน</details>
- Backtracking
  - เป็นการเก็บคำตอบย้อนหลัง หลังจากที่เราเข้าไปข้างอย่างแล้ว
  - [https://grader.mwit.ac.th/problem/walking\\_bot\\_2](https://grader.mwit.ac.th/problem/walking_bot_2)<sup>◦</sup>
- Branch and Bound (implementation)
  - <https://grader.mwit.ac.th/problem/snakeword><sup>◦</sup>
- Challenge
  - <https://grader.mwit.ac.th/problem/teleport><sup>◦</sup>  
hint</summary>ad-hoc, implementation</details>

—

- Eulerian
- Hamiltonian

—

- Colorings
- Connectivity check
- Finding cycles visualgo<sup>◦</sup>
- Bipartiteness check

### 23.2. Day 2

[visualgo](<https://visualgo.net/en/sssp?slide=1>)<sup>◦</sup>

- Dijkstra's Algorithm (implementation)
  - <https://grader.mwit.ac.th/problem/turboprogramming><sup>◦</sup>  
hint</summary>dijkstra ตรงๆ</details>
  - <https://grader.mwit.ac.th/problem/town><sup>◦</sup>  
hint</summary>dijkstra ไม่ตรงมาก</details>
  - <https://grader.mwit.ac.th/problem/followpeatt><sup>◦</sup>  
hint</summary>dijkstra แบบมีเงื่อนไข</details>

- [https://grader.mwit.ac.th/problem/toi14\\_logistics](https://grader.mwit.ac.th/problem/toi14_logistics)°

hint</summary>dijkstra ซับซ้อนขึ้น</details>

- Bellman-Ford
  - Negative cycles
- Floyd-Warshall (implementation)
  - [https://grader.mwit.ac.th/problem/toi17\\_1221](https://grader.mwit.ac.th/problem/toi17_1221)°

hint</summary>all-pair</details>

- Johnson (exist)
- A\* Search

—

visualgo°

- Kruskal's (implementation)
  - Union-find
  - <https://grader.mwit.ac.th/problem/mst>°

hint</summary>ตรงสุดๆ</details>

- Prim's

—

### 23.3. Day 3

- Diameter
- Lowest common ancestor (implementation)
- Euler Tour Technique
- [https://grader.mwit.ac.th/problem/toi12\\_weakpoint](https://grader.mwit.ac.th/problem/toi12_weakpoint)° hint</summary>one-cycle, dp?</details>

—

- [https://grader.mwit.ac.th/problem/toi14\\_technology](https://grader.mwit.ac.th/problem/toi14_technology)°
- Kosaraju's
- Challenge
  - [https://grader.mwit.ac.th/problem/walk\\_around](https://grader.mwit.ac.th/problem/walk_around)°

hint</summary>cycle, union find, reverse query</details>

### 23.4. เพิ่มเติม

- Principles of Algorithmic Problem Solving
  - <https://usaco.guide/PAPS.pdf#page=215>°
- Competitive Programmer's Handbook
  - <https://usaco.guide/CPH.pdf#page=119>°

- Algorithms for Competitive Programming

- <https://cp-algorithms.com/#graphs>

- ตะลุยโจทย์ Graph ระดับโหดใน Competitive Programming (aquablitz11)

[ตะลุยโจทย์ Graph ระดับโหดใน Competitive Programming](<http://tcpc.me/2019/08/19/state-graph-tutorial.html>)

- การบรรยายวิชา การออกแบบและวิเคราะห์อัลกอริทึม โดย สมชาย ประสิทธิ์จิ๋วตระกูล
  - [อัลกอริทึม : 6.1 กราฟ - นิยามและการใช้งาน](<https://youtu.be/3brx7JpABNg>)
  - [อัลกอริทึม : 6.2 กราฟ - การแทนกราฟ](<https://youtu.be/cnkRY67HLM0>)
  - [อัลกอริทึม : 6.3 กราฟ - การค้นหาตามแนวกว้าง](<https://youtu.be/keMPqNG-8JA>)
  - [อัลกอริทึม : 6.4 กราฟ - การค้นหาตามแนวลึก](<https://youtu.be/EdNOAGyCps>)
  - [อัลกอริทึม : 6.5 กราฟ - การเรียงลำดับทอพอโลยี](<https://youtu.be/Vfc0FgeLd7A>)
  - [อัลกอริทึม : 6.6 กราฟ - การเชื่อมต่อแบบเข้ม](<https://youtu.be/NEKOJbOYLHA>)
  - [อัลกอริทึม : 6.7 กราฟ - Prim : ต้นไม้แบบทอดข้ามต่ำสุด](<https://youtu.be/QTqs56rXg9U>)
  - [อัลกอริทึม : 6.8 กราฟ - Kruskal : ต้นไม้แบบทอดข้ามต่ำสุด]([https://youtu.be/6TkX\\_sFOq5E](https://youtu.be/6TkX_sFOq5E))
  - [อัลกอริทึม : 6.9 กราฟ - Dijkstra : วิธีสั้นสุดแหล่งต้นทางเดียว](<https://youtu.be/bBkrRrrSX2M>)
  - [อัลกอริทึม : 6.10 กราฟ - Bellman-Ford : วิธีสั้นสุดแหล่งต้นทางเดียว](<https://youtu.be/Ypckss0NJm4>)
  - [อัลกอริทึม : 6.11 กราฟ - Floyd-Warshall : วิธีสั้นสุดของทุกคู่ปม](<https://youtu.be/8AnC3Z9LwKY>)
  - [อัลกอริทึม : 6.12 กราฟ - Johnson : วิธีสั้นสุดของทุกคู่ปม](<https://youtu.be/8deyv0u0hyE>)

## 24. Topological sorting (implementation)

DAG - Directed Acyclic Graph

[<https://cp-algorithms.com/graph/topological-sort.html>](<https://cp-algorithms.com/graph/topological-sort.html>)

```
int n; // number of vertices
vector<vector<int>> adj; // adjacency list of graph
vector<bool> visited;
vector<int> ans;

void dfs(int v) {
 visited[v] = true;
 for (int u : adj[v]) {
 if (!visited[u])
 dfs(u);
 }
 ans.push_back(v);
}

void topological_sort() {
 visited.assign(n, false);
 ans.clear();
 for (int i = 0; i < n; ++i) {
 if (!visited[i])
 dfs(i);
 }
 reverse(ans.begin(), ans.end());
}
```

- [[https://grader.mwit.ac.th/problem/toi14\\_technology](https://grader.mwit.ac.th/problem/toi14_technology)]([https://grader.mwit.ac.th/problem/toi14\\_technology](https://grader.mwit.ac.th/problem/toi14_technology))



## 25. Paths and circuits

### 25.1. Eulerian

#### 25.1.1. Path

เป็น path ที่ท่องเที่ยวไปในเส้นเชื่อมทุกเส้นเพียงหนึ่งครั้งเท่านั้น

#### 25.1.2. Circuit

เป็น Eulerian path ที่เริ่มและจบที่โหนดเดียวกัน

#### 25.1.3. Existence

เงื่อนไขที่จะมี Eulerian path สำหรับ

- กราฟไม่มีทิศทาง
  - ดีกรีของทุกโหนดเป็นจำนวนคู่ หรือ
  - ดีกรีของสองโหนดเป็นจำนวนคี่

🧠 เพราะอะไร?

</aside>

- กราฟมีทิศทาง
  - ในแต่ละโหนด ดีกรีเข้าจะต้องเท่ากับดีกรีออก หรือ
  - มีโหนดหนึ่งมีดีกรีเข้ามากกว่าดีกรีออกเท่ากับหนึ่ง อีกโหนดหนึ่งมีดีกรีออกมากกว่าดีกรีเข้าเท่ากับหนึ่ง และโหนดที่เหลือมีดีกรีเข้าเท่ากับดีกรีออก

#### 25.1.4. Hierholzer's algorithm

### 25.2. Hamiltonian

เป็น path ที่ท่องเที่ยวไปในทุกโหนดเพียงหนึ่งครั้งเท่านั้น

#### 25.2.1. Existence

ไม่มี efficient method เนื่องจากยังเป็นปัญหาประเภท NP-hard

## 26. Weighted Shortest Path

[visualgo](https://visualgo.net/en/sssp?slide=1<sup>o</sup>)

### 26.0.1. Dijkstra's Algorithm (implementation)

[https://cp-algorithms.com/graph/dijkstra.html<sup>o</sup>](https://cp-algorithms.com/graph/dijkstra.html<sup>o</sup>)

```
const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> & d, vector<int> & p) {
 int n = adj.size();
 d.assign(n, INF);
 p.assign(n, -1);
 vector<bool> u(n, false);

 d[s] = 0;
 for (int i = 0; i < n; i++) {
 int v = -1;
 for (int j = 0; j < n; j++) {
 if (!u[j] && (v == -1 || d[j] < d[v]))
 v = j;
 }

 if (d[v] == INF)
 break;

 u[v] = true;
 for (auto edge : adj[v]) {
 int to = edge.first;
 int len = edge.second;

 if (d[v] + len < d[to]) {
 d[to] = d[v] + len;
 p[to] = v;
 }
 }
 }
}
```

- [https://grader.mwit.ac.th/problem/turboprogramming<sup>o</sup>](https://grader.mwit.ac.th/problem/turboprogramming<sup>o</sup>)

dijkstra ตัวอย่าง

```
// credit Mok MWIT29
#include <bits/stdc++.h>
#define ii pair<int, int>
using namespace std;
vector<ii> adj[100100];
int dist[100100];
int main() {
 int N, M, Q;
 scanf("%d %d %d", &N, &M, &Q);
```

```

for (int i = 1, u, v, w; i ≤ M; i++) {
 scanf("%d %d %d", &u, &v, &w);
 adj[u].push_back(make_pair(v, w));
 // adj[v].push_back(make_pair(u, w));
}
for (int i = 1; i ≤ N; i++)
 dist[i] = INT_MAX;
priority_queue<ii, vector<ii>, greater<ii>> pq;
dist[1] = 0;
pq.push(make_pair(0, 1));
while (!pq.empty()) {
 int d = pq.top().first;
 int n = pq.top().second;
 pq.pop();
 for (auto x : adj[n]) {
 if (dist[x.first] > d + x.second) {
 dist[x.first] = d + x.second;
 pq.push(make_pair(d + x.second, x.first));
 }
 }
}
while (Q--) {
 int a;
 scanf("%d", &a);
 if (dist[a] == INT_MAX)
 printf("-1\n");
 else
 printf("%d\n", dist[a]);
}
}

```

- [<https://grader.mwit.ac.th/problem/town>](<https://grader.mwit.ac.th/problem/town>)

dijkstra ไม่ตรงมาก

- [<https://grader.mwit.ac.th/problem/followpeatt>](<https://grader.mwit.ac.th/problem/followpeatt>)

dijkstra แบบมีเงื่อนไข

- [[https://grader.mwit.ac.th/problem/toi14\\_logistics](https://grader.mwit.ac.th/problem/toi14_logistics)]([https://grader.mwit.ac.th/problem/toi14\\_logistics](https://grader.mwit.ac.th/problem/toi14_logistics))

dijkstra ซ้ำซ้อนขึ้น

## 26.0.2. Bellman-Ford

- Negative cycles

```

// credit from https://github.com/Autoratch/practice
#include <bits/stdc++.h>
using namespace std;

int n, m, s, e;
vector<pair<int, pair<int, int>>> adj;

```

```

vector<int> dist;

int main() {
 ios_base::sync_with_stdio(0);
 cin.tie(0);

 cin >> n >> m >> s >> e;

 adj.resize(m);
 dist.assign(n, INT_MAX);

 for (int i = 0; i < m; i++) {
 int a, b, d;
 cin >> a >> b >> d;
 adj[i] = {d, {a, b}};
 }

 dist[0] = 0;
 for (int i = 0; i < n - 1; i++)
 for (int j = 0; j < m; j++) {
 int a = adj[i].second.first, b = adj[i].second.second, d = adj[i].first;
 if (dist[a] != INT_MAX and dist[a] + d < dist[b])
 dist[b] = dist[a] + d;
 }

 cout << dist[e];
}

```

### 26.0.3. Floyd-Warshall (implementation)

[<https://cp-algorithms.com/graph/all-pair-shortest-path-floyd-warshall.html>°]  
 (https://cp-algorithms.com/graph/all-pair-shortest-path-floyd-warshall.html°)

```

for (int k = 0; k < n; ++k) {
 for (int i = 0; i < n; ++i) {
 for (int j = 0; j < n; ++j) {
 d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
 }
 }
}

```

- [[https://grader.mwit.ac.th/problem/toi17\\_1221](https://grader.mwit.ac.th/problem/toi17_1221)°](https://grader.mwit.ac.th/problem/toi17\_1221°)

all-pair

### 26.0.4. Johnson (exist)

### 26.0.5. A\* Search

[<https://www.redblobgames.com/pathfinding/a-star/introduction.html>°](https://www.redblobgames.com/pathfinding/a-star/introduction.html°)

## 27. Strong connectivity

[<https://cp-algorithms.com/graph/strongly-connected-components.html>°](<https://cp-algorithms.com/graph/strongly-connected-components.html>°)

### 27.0.1. Kosaraju's

```
// credit from https://github.com/Autoratch/practice
#include <bits/stdc++.h>
using namespace std;

const int N = 1e5 + 1;

int n, m;
vector<int> adj[N], rev[N];
stack<int> st;
bool visited[N];

void dfs(int u) {
 if (visited[u])
 return;
 visited[u] = true;
 for (int v : adj[u])
 dfs(v);
 st.push(u);
}

void scc(int u) {
 if (visited[u])
 return;
 visited[u] = true;
 cout << u << ' ';
 for (int v : rev[u])
 scc(v);
}

int main() {
 ios_base::sync_with_stdio(0);
 cin.tie(0);

 cin >> n >> m;

 for (int i = 0; i < m; i++) {
 int a, b;
 cin >> a >> b;
 adj[a].push_back(b);
 rev[b].push_back(a);
 }

 for (int i = 1; i ≤ n; i++)
 if (!visited[i])
 dfs(i);

 memset(visited, 0, sizeof visited);

 while (!st.empty()) {
```

```

 int u = st.top();
 st.pop();
 if (visited[u])
 continue;
 scc(u);
 cout << '\n';
}
}

```

### 27.0.2. Challenge

- [[https://grader.mwit.ac.th/problem/walk\\_around](https://grader.mwit.ac.th/problem/walk_around)]([https://grader.mwit.ac.th/problem/walk\\_around](https://grader.mwit.ac.th/problem/walk_around))

## 28. Recap Graph Day 2

### 28.1. Weighted Shortest Path

- Dijkstra's Algorithm (implementation)

```
if(dist[n] + x.second < dist[x.first]){
 dist[x.first] = dist[n] + x.second;
}
```

- Bellman-Ford
  - Negative cycles
- Floyd-Warshall (implementation)

```
for(int k = 0; k < N; k++){
 for(int i = 0; i < N; i++){
 for(int j = 0; j < N; j++){
 if(dist[i][k] == INT_MAX || dist[k][j] == INT_MAX) continue;
 dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
 }
 }
}
```

- Johnson (exist)
- A\* Search

🧠 heuristic

</aside>

—

### 28.2. Minimum Spanning Tree

- Kruskal's (implementation)
  - Union-find

```
sort(v.begin(), v.end());
```

```
int ans = 0;
for(auto x : v){
 int hx = findpa(x.second.first), hy = findpa(x.second.second);
 if(hx != hy){
 ans += x.first;
 pa[hx] = hy;
 }
}
```

- Prim's

—

### 28.3. Proof

[<https://cp-algorithms.com/graph/dijkstra.html>](<https://cp-algorithms.com/graph/dijkstra.html>)

[[https://wcipeg.com/wiki/Floyd-Warshall\\_algorithm](https://wcipeg.com/wiki/Floyd-Warshall_algorithm)]([https://wcipeg.com/wiki/Floyd%E2%80%93Warshall\\_algorithm](https://wcipeg.com/wiki/Floyd%E2%80%93Warshall_algorithm))

[[https://cp-algorithms.com/graph/mst\\_kruskal.html](https://cp-algorithms.com/graph/mst_kruskal.html)]([https://cp-algorithms.com/graph/mst\\_kruskal.html](https://cp-algorithms.com/graph/mst_kruskal.html))



## 29. Minimum Spanning Tree

[visualgo](https://visualgo.net/en/mst?slide=1°)

### 29.1. Kruskal's (implementation)

- Union-find

```
// credit from https://github.com/Autoratch/practice
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
#define MOD 1e9 + 7
#define pii pair<int, pair<int, int>>

int n, m;
vector<int> pa;
priority_queue<pii, vector<pii>, greater<pii>> q;

int root(int x) {
 if (pa[x] == x)
 return x;
 else
 return pa[x] = root(pa[x]);
}

int kruskal() {
 int ans = 0;

 pa.resize(n);
 for (int i = 0; i < n; i++)
 pa[i] = i;

 while (!q.empty()) {
 int w = q.top().first, x = q.top().second.first, y = q.top().second.second;
 q.pop();
 if (root(x) == root(y))
 continue;
 ans += w;
 pa[root(x)] = pa[root(y)];
 }

 return ans;
}

int main() {
 ios_base::sync_with_stdio(0);
 cin.tie(0);

 cin >> n >> m;

 for (int i = 0; i < m; i++) {
 int a, b, d;
 cin >> a >> b >> d;
 q.push({d, {a, b}});
 }
}
```

```

 cout << kruskal();
}

```

## 29.2. Prim's

```

// credit from https://github.com/Autoratch/practice
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
#define MOD 1e9 + 7

int n, m, ans;
vector<bool> visited;
vector<vector<pair<int, int>>> adj;
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>>
 q;

int main() {
 ios_base::sync_with_stdio(0);
 cin.tie(0);

 cin >> n >> m;

 adj.resize(n);
 visited.resize(n);

 for (int i = 0; i < m; i++) {
 int a, b, d;
 cin >> a >> b >> d;
 adj[a].push_back({d, b});
 adj[b].push_back({d, a});
 }

 q.push({0, 0});

 while (!q.empty()) {
 int w = q.top().first, p = q.top().second;
 q.pop();
 if (visited[p])
 continue;
 visited[p] = true;
 ans += w;
 for (int i = 0; i < adj[p].size(); i++)
 if (!visited[adj[p][i].second])
 q.push(adj[p][i]);
 }

 cout << ans;
}

```

## 30. Graph Applications

### 30.1. Colorings

สามารถใช้การ search ต่าง ๆ เช่น dfs ในการระบายสีโหนดได้

ปัญหาการระบายสีโหนดให้โหนดที่ติดกันมีสีต่างกันเสมอเป็นปัญหาที่พบเจอได้บ่อยในสาขา คอมพิวเตอร์

### 30.2. Connectivity check

DFS แล้วเช็ค ว่า visit ครบทุกโหนดหรือไม่

### 30.3. Finding cycles

[visualgo](<https://visualgo.net/en/cyclefinding?slide=1>)

ถ้า component มี  $c$  โหนด และไม่มี cycle เลยแล้ว component นั้นจะต้องมีเส้นเชื่อมทั้งหมด  $c - 1$  เส้น และเป็นกราฟต้นไม้

### 30.4. Bipartiteness check

กราฟ bipartite เป็นกราฟเมื่อสามารถใช้สีสองสีในการระบายสีโหนดโดยทุกโหนดที่ติดกันต้องมีสีที่แตกต่างกัน ตัวอย่างกราฟที่ไม่เป็น bipartite คือ

[<https://cp-algorithms.com/graph/bipartite-check.html>](<https://cp-algorithms.com/graph/bipartite-check.html>)

## 31. Tree algorithms

### 31.1. Diameter

- Algorithm 1
  - ตั้ง node มั่ว ๆ มาเป็น root
  - หา path ยาวสุดของแต่ละลูก แล้วหาเส้นผ่านศูนย์กลาง
- Algorithm 2
  - เลือกโหนดมั่ว ๆ เป็นโหนด a
  - หาโหนดที่อยู่ไกลที่สุดจากโหนด a เป็นโหนด b
  - หาโหนดที่อยู่ไกลที่สุดจากโหนด b เป็นโหนด c
  - จะได้เส้นผ่านศูนย์กลางคือเส้นทางจาก b ไป c

### 31.2. Lowest common ancestor (implementation)

// credit from <https://github.com/Autoratch/practice>

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
const int N = 1e5 + 1;
```

```
int n, q;
```

```
int dp[21][N], lv[N];
```

```
vector<int> adj[N];
```

```
void dfs(int u, int p, int l) {
 dp[0][u] = p, lv[u] = l;
 for (int v : adj[u])
 dfs(v, u, l + 1);
}
```

```
int lca(int a, int b) {
 if (lv[a] < lv[b])
 swap(a, b);
 for (int i = 20; i ≥ 0; i--)
 if (lv[dp[i][a]] ≥ lv[b])
 a = dp[i][a];
 if (a == b)
 return a;
 for (int i = 20; i ≥ 0; i--)
 if (dp[i][a] ≠ dp[i][b])
 a = dp[i][a], b = dp[i][b];
 return dp[0][a];
}
```

```
int main() {
 ios_base::sync_with_stdio(0);
 cin.tie(0);

 cin >> n;

 for (int i = 0; i < n - 1; i++) {
 int a, b;
 cin >> a >> b;
```

```

 adj[a].push_back(b);
}

dfs(0, 0, 1);

for (int i = 1; i ≤ 20; i++)
 for (int j = 1; j ≤ n; j++)
 dp[i][j] = dp[i - 1][dp[i - 1][j]];

cin >> q;

while (q--) {
 int a, b;
 cin >> a >> b;
 cout << lca(a, b) << '\n';
}
}

```

### 31.3. Euler Tour Technique

[<https://usaco.guide/gold/tree-euler?lang=cpp>](<https://usaco.guide/gold/tree-euler?lang=cpp>)

[<https://cses.fi/problemset/task/1137>](<https://cses.fi/problemset/task/1137>)

### 31.4. Challenge

- [[https://grader.mwit.ac.th/problem/toi12\\_weakpoint](https://grader.mwit.ac.th/problem/toi12_weakpoint)]([https://grader.mwit.ac.th/problem/toi12\\_weakpoint](https://grader.mwit.ac.th/problem/toi12_weakpoint))

## 32. แนวทางการฝึกด้วยตัวเอง

เนื่องจากค่าย สอวน. เป็นแนวทางหลักในเส้นทางการเขียนโปรแกรมเชิงแข่งขัน เลยจะอ้างอิงเนื้อหาตามลำดับของค่าย สอวน. เป็นช่วง ๆ ดังนี้

### 32.1. สำหรับก่อน สอวน. ค่าย 1

ถ้าหากไม่เคยเขียนโปรแกรมมาก่อนเลย แนะนำให้เริ่มศึกษาภาษา C++ จากหนังสือ C++ Language Tutorial<sup>1</sup> ก่อน แล้วหลังจากนั้นใช้หนังสือ Competitive Programmer's Handbook<sup>2</sup>

### 32.2. สำหรับช่วงหลัง สอวน. ค่าย 1

### 32.3. สำหรับช่วงหลัง สอวน. ค่าย 2

### 32.4. สำหรับขั้นต่อไป

---

<sup>1</sup><https://cplusplus.com/files/tutorial.pdf><sup>o</sup>

<sup>2</sup><https://cses.fi/book/book.pdf><sup>o</sup>

## 33. Resources

- แนะนำให้หนังสือเล่มนี้เป็นหลัก<sup>3</sup>
- หากไม่เข้าใจบางหัวข้อ ต้องการอ่านเพิ่มเติมแนะนำ websites
  - USACO Guide<sup>4</sup>
  - <https://cp-algorithms.com/><sup>◦</sup>
- เว็บไซต์สำหรับทำโจทย์
  - <https://cses.fi/problemset/><sup>◦</sup> แหล่งเดียวกับหนังสือหลัก
  - <https://programming.in.th/><sup>◦</sup> เว็บหลักในไทย
  - <https://otog.in.th/><sup>◦</sup> เว็บที่นักเรียน สอวน. ช่วยกันทำขึ้นมาเอง
  - <https://codeforces.com/><sup>◦</sup> เว็บหลักในต่างประเทศ
- หนังสือเพิ่มเติมเนื้อหาจะซ้ำซ้อนกับด้านบน <https://usaco.guide/PAPS.pdf><sup>◦</sup>
- ถ้าสามารถหาได้คร่าว ๆ ว่าในค่ายใช้แหล่งไหนเป็นหลัก แนะนำให้เตรียมตัวด้วยแหล่งที่แตกต่างกัน จะได้ทำโจทย์ครอบคลุมหลากหลายแนวกว่าคนในค่าย
- ข้อสอบของแต่ละศูนย์ค่าย สอวน. จะมีแนวเป็นของตัวเอง อันนี้ต้องหาถามรุ่นพี่ดูครับ

### 33.1. Recommended Books

- [C++ Language Tutorial](<https://cplusplus.com/files/tutorial.pdf>)<sup>◦</sup>
- [Competitive Programmer's Handbook (in C++)](<https://cses.fi/book/book.pdf>)<sup>◦</sup>
- [Principles of Algorithmic Problem Solving (in C++)](<https://usaco.guide/PAPS.pdf>)<sup>◦</sup>

### 33.2. Recommended Websites

- [beta.programming.in.th](<https://beta.programming.in.th/>)<sup>◦</sup>
- [CSES](<https://cses.fi/problemset/>)<sup>◦</sup>
- [Codeforces](<https://codeforces.com/>)<sup>◦</sup>
- [Kattis](<https://open.kattis.com/>)<sup>◦</sup>
- [AtCoder](<https://atcoder.jp/>)<sup>◦</sup>
- [Kenkoooo](<https://kenkoooo.com/atcoder#/table/>)<sup>◦</sup>
- [CP Algorithms](<https://cp-algorithms.com/>)<sup>◦</sup>

### 33.3. Random Posts

- [C++11 for programming contests...](<https://codeforces.com/blog/entry/10124>)<sup>◦</sup>
- [C++ Tricks](<https://codeforces.com/blog/entry/15643>)<sup>◦</sup>

---

<sup>3</sup><https://cses.fi/book/book.pdf><sup>◦</sup>

<sup>4</sup><https://usaco.guide/><sup>◦</sup>